

VOLUME 13 / RUST BACKEND MASTERY

完全コード — 演習・解答・用語集・索引

RustNote の全ファイルを順に読み、自分の PC で同じ結果を出して確かめる最終巻

第13巻: 完全コード — 演習・解答・用語集・索引

この巻で身に付けること

第12巻までで、RustNote を設計し（第9巻）、作り（第10巻）、確かめ（第11巻）、公開する（第12巻）までの流れを学びました。どの巻も、説明に必要な部分だけを抜き出して見せています。だから、「全部をつなげると、結局どのファイルに何が書いてあるのか」は、まだぼんやりしているかもしれません。

この巻は、教材のフォルダに入っている**完成した RustNote のソース**（`Rust/rustnote/`）を、**全部のファイルについて**、上から順に読んでいく巻です。ただ貼り付けて眺めるのではなく、ファイルごとに次の3つを確かめます。

- そのファイルは、**何を守るために**あるのか（どんな事故を防いでいるのか）
- **誰が読む**のか（cargo・rustup・rustc・ブラウザ・Docker・Fly.io など）
- 書き間違えたり消したりすると、**何が起きる**のか（実際のエラーで確かめる）

巻の最後には、全巻をまたぐ総合演習と解答、全巻の用語集、索引を置きます。

到達目標

この巻を終えると、次のことができるようになります。

- RustNote の 42 個のファイルについて、それぞれの役割と、読む道具（cargo・rustc・Docker など）を説明できる
- 教材と同じ版の道具と依存クレートで、RustNote を自分の PC でゼロからビルドし、テスト・整形・Clippy の確かめを通せる
- 1つのファイルを書き換えたときに、どのコマンドの、どんなエラーで気づけるかを予想し、実際に確かめられる
- 総合演習で、新しい機能を足すときの置き場所と、壊していないことの確かめ方を、自分で決められる

学習地図

10 個の単元で、RustNote のファイルを「土台から上へ」の順に読みます。最初は版を決める設定ファイル、次に DB の表、起動、型、保存、ログイン、画面、見た目、最後にテストと公開です。

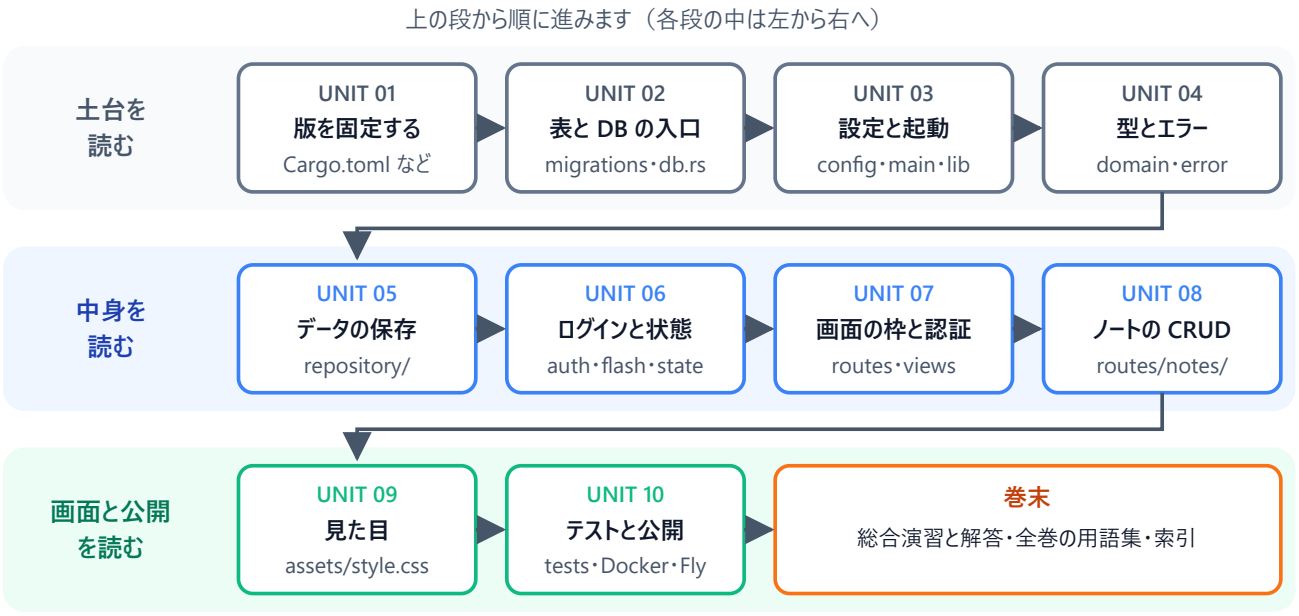


図13-1: 第13巻の学習地図。土台の設定ファイルから読み始め、型・保存・ログイン・画面を経て、テストと公開で終わる。巻末に総合演習・用語集・索引を置く

単元	テーマ	読むファイル
UNIT 01	版を固定する設定ファイル	`Cargo.toml`・`Cargo.lock`・`rust-toolchain.toml`・`rustfmt.toml`・`.gitignore`・`.env.example`
UNIT 02	表を作る SQL と DB の入口	`migrations/0001〜0003`・`src/db.rs`
UNIT 03	設定と起動の順番（準備中）	`src/config.rs`・`src/main.rs`・`src/lib.rs`
UNIT 04	ドメインの型とエラーの分類（準備中）	`src/domain.rs`・`src/error.rs`
UNIT 05	リポジトリ（準備中）	`src/repository.rs`・`src/repository/` の3つ
UNIT 06	ログイン・お知らせ・共有の状態（準備中）	`src/auth.rs`・`src/flash.rs`・`src/state.rs`
UNIT 07	画面の枠・部品・登録とログインの画面（準備中）	`src/routes.rs`・`src/views.rs`・`src/routes/` の signup・login・logout・health
UNIT 08	ノートの CRUD（準備中）	`src/routes/notes.rs`・`src/routes/notes/` の4つ
UNIT 09	見た目（準備中）	`assets/style.css`
UNIT 10	テスト・Docker・Fly.io・CI（準備中）	`tests/` の3つ・`Dockerfile`・`.dockerignore`・`fly.toml`・`ci.yml`・`README.md`

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03〜10 と、巻末の総合演習・解答・全巻の用語集・索引は準備中で、次の更新から順に追加します。

RustNote の全ファイル

RustNote は、次の 42 個のファイルでできています（`git ls-files` で数えた、Git で管理しているファイルです）。行数は `Cargo.lock` を除くと約 4,600 行です。

ファイル	行数	単元	ひとことで言うと
`Cargo.toml`	35	01	使うクレートとその版の一覧
`Cargo.lock`	3,753	01	実際に選ばれた全クレートの版と、中身の指紋（自動で作られる）
`rust-toolchain.toml`	5	01	このフォルダで使う Rust の版（1.99.1）
`rustfmt.toml`	2	01	`cargo fmt` の設定
`.gitignore`	15	01	Git に入れないファイル（ビルドの成果物・DB・`.env`）
`.env.example`	15	01	環境変数の見本
`migrations/0001_users.sql` ほか2つ	9・18・12	02	表を作る SQL
`src/db.rs`	185	02	DB への接続とマイグレーション
`src/config.rs`・`src/main.rs`・`src/lib.rs`	182・54・27	03	設定の読み込み・起動の入口・部品の一覧
`src/domain.rs`・`src/error.rs`	518・152	04	検査済みの型と、エラーの分類
`src/repository.rs` と `src/repository/` の3つ	39・243・85・112	05	ノート・ユーザー・セッションの読み書き
`src/auth.rs`・`src/flash.rs`・`src/state.rs`	179・117・42	06	ログイン・一度だけのお知らせ・共有の状態
`src/routes.rs`・`src/views.rs`	142・249	07	ルーターとレイアウト・画面の部品
`src/routes/` の signup・login・logout・health	162・89・22・34	07	登録・ログイン・ログアウト・ヘルスチェック
`src/routes/notes.rs` と `src/routes/notes/` の4つ	145・80・113・29・27	08	ノートの一覧・作成・詳細・編集・完了の切り替え・削除
`assets/style.css`	290	09	画面のスタイル
`tests/common/mod.rs`・`tests/http.rs`・`tests/repository.rs`	159・398・248	10	テストの道具と、41 件のテストのうち 22 件
`Dockerfile`・`.dockerignore`・`fly.toml`・`ci.yml`・`README.md`	50・7・38・48・192	10	コンテナ・公開・CI の設定と説明書

- 残りの 19 件のテストは、`src/` の中のファイルの最後にある `#[cfg(test)]` のユニットテストです。それぞれのファイルの単元で読みます。

- 名前が `.` で始まるファイル（`.gitignore` など）は、Windows の PowerShell やエクスプローラーではそのまま表示されます。macOS や Linux の `ls` では隠れるので、`ls -a` で表示します。

前提知識（先に読んでおく巻）

この巻は、全巻のまとめです。どの単元も、元になった巻を「第9巻 UNIT 04」のように示します。

- **第1巻（環境構築）**: Rust・VS Code・Git が使えること。ARM64 版の Windows では、LLVM（clang）も入れておきます（第9巻の巻頭にも手順があります）。
- **第2巻〜第5巻**: Rust の文法、所有権、型と trait とエラー、Cargo と features（第4巻 UNIT 07）、async。
- **第6巻〜第8巻**: Topcoat、Turso・libSQL と SQL、ログインとセッション。
- **第9巻〜第12巻**: RustNote の設計・実装・品質の確かめ・公開。この巻は、その完成形を読む巻です。

> **注意**: 第12巻は、書き直しの途中です。この巻の本文は、教材のフォルダの `Rust/rustnote/` のコードと、この巻で実際に動かした結果だけをもとに書いているので、ほかの巻が未完成でも読み進められます。

この巻の進め方

完成したコードを読むときは、つい「読んで分かったつもり」になりがちです。各単元では、次のサイクルで進めてください。

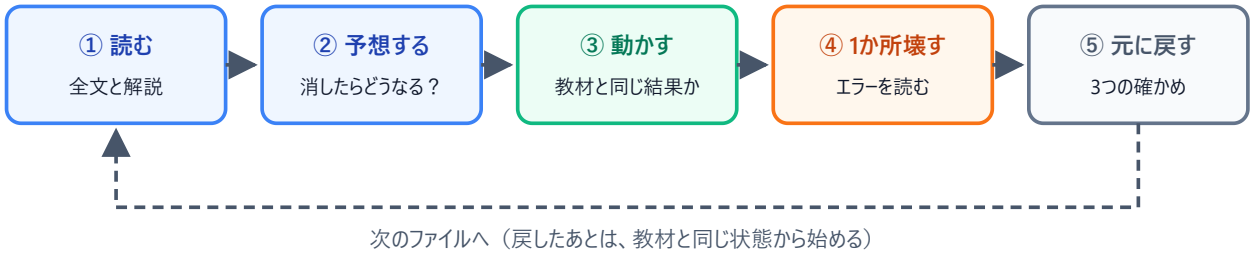


図13-2: この巻の進め方。読んで、予想して、動かして、1か所だけ壊してエラーを読み、元に戻してから次のファイルへ進む

1. **読む**: ファイルの全文と解説を読みます。
2. **予想する**: 「この行を消したら、どのコマンドが、どんなエラーを出すか」を、先に予想します。
3. **動かす**: 教材のコマンドをそのまま実行し、結果が教材と同じか比べます。
4. **1か所だけ壊す**: 「よくあるエラーと読み方」の節のとおりに、1か所だけ書き換えて、エラーを自分の目で見ます。
5. **元に戻す**: 書き換えたファイルを元に戻し、次の3つのコマンドが全部通ることを確かめてから、次のファイルへ進みます。

```
# 見た目が Rust の標準の形か（直すときは --check を外す）
cargo fmt --check
# よくある書き間違いを、警告ではなくエラーとして止める。--locked は Cargo.lock を書き換えさせない
cargo clippy --all-targets --locked -- -D warnings
# テストを全部実行する
cargo test --locked
```

この3つは、RustNote の `ci.yml`（GitHub Actions の設定の例）で、コードを送るたびに自動で実行されるのと同じコマンドです（UNIT 10（準備中））。

> **ヒント:** 元に戻すのがいちばん確実なのは、教材のフォルダから、そのファイルだけをもう一度コピーすることです。練習用のフォルダを Git で管理しておく（`git init` して最初に全部をコミットしておく）と、`git diff` で変えた所を確かめ、`git checkout -- ファイル名` で元に戻せます（UNIT 01 の問3）。

使うツールと版

この巻のコードは、すべて次の環境で実際にビルド・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.99 以上**（本書シリーズの基準。Topcoat 0.11 が 1.99 以上を求めます）。動作確認は rustc 1.99.1 / cargo 1.99.1（`aarch64-pc-windows-msvc`）
edition	2024
RustNote	教材の `Rust/rustnote/`（`rustnote` 0.1.0）。この巻では中身を変えずに読む
直接使うクレート	topcoat 0.11.0、libsql 0.10.0、tokio 1.54.1、argon2 0.6.0、async-trait 0.1.92、serde 1.0.229、thiserror 2.0.21、anyhow 1.0.104、dotenvy 0.15.7、tempfile 3.27.0（テスト用）
topcoat-cli	0.11.0（UNIT 09・10 で使います。UNIT 01 では使いません）

練習用のフォルダを用意する

第9巻と同じように、教材の `Rust/rustnote/` を**作業用のフォルダにコピー**して使います。この巻では、わざとファイルを書き換えてエラーを見るので、お手本そのものには触らないことが特に大切です。

```
# 教材の rustnote フォルダを、中身ごと（-Recurse）作業用のフォルダへコピーする
Copy-Item -Recurse "教材のフォルダ\Rust\rustnote" .\rustnote
cd rustnote
# このフォルダで使われる Rust の版を確かめる
rustup show active-toolchain
```

```
1.99.1-aarch64-pc-windows-msvc (overridden by '...\rustnote\rust-toolchain.toml')
```

- `"教材のフォルダ\Rust\rustnote"` の部分は、自分の PC で教材を置いた場所に読み替えてください。
- 第9巻で使ったコピーがあれば、それとは別の、新しいコピーを作ってください。第9巻のコピーには、確かめ用のテストを足してあるからです。

UNIT 01: 版を固定する設定ファイル

この単元のゴール

- `Cargo.toml`・`Cargo.lock`・`rust-toolchain.toml`・`rustfmt.toml`・`.gitignore`・`.env.example` の6つのファイルについて、それぞれ誰が読み、何を決めているかを説明できる
- `"=1.53.3"` と `"1.53.3"` の版の書き方の違いと、`Cargo.lock` と `--locked` が「誰の PC でも同じ物を作る」ために何をしているかを説明できる
- features（クレートの機能のスイッチ）は、依存のつながり全体で**足し算**されることを、実際の出力で確かめられる
- 版が古い・機能の名前の打ち間違い・`Cargo.lock` の食い違いなどのエラーを読み、どのファイルを直せばよいかを判断できる

たとえ話でつかむ

料理教室で、全員に同じケーキを焼いてもらう場面を考えます。先生は、次のような紙を配ります。

- **材料表**には「小麦粉 A 社の 1.53.3 番」「卵 M サイズ」のように、銘柄と規格まで書いてあります。「小麦粉 適量」では、人によって違うケーキになるからです。
- **買い物のレシート**には、実際に買った品物が全部、ロット番号つきで並んでいます。材料表に書いていない、小麦粉の袋の中に入っていた乾燥剤まで載っています。次に買うときも、同じロットを選べます。
- **オーブンの型番**も決めておきます。型番が古いオーブンでは、焼けない温度があるからです。
- **盛り付けの決まり**があれば、誰が作っても同じ見た目になります。
- **持ち帰らない物の札**を、生ごみや、家の鍵に付けておきます。
- **鍵の置き場所のメモ**には「玄関の鍵は、ここに置く」とだけ書きます。鍵そのものは、紙に貼りません。

RustNote の6つの設定ファイルは、この紙と同じ役目をしています。

料理教室	RustNote のファイル	読むもの
材料表（銘柄と規格）	`Cargo.toml`（使うクレートと版）	cargo
買い物のレシート（ロット番号つき）	`Cargo.lock`（選ばれた全クレートの版と指紋）	cargo
オーブンの型番	`rust-toolchain.toml`（Rust の版）	rustup
盛り付けの決まり	`rustfmt.toml`（整形の設定）	rustfmt（`cargo fmt`）
持ち帰らない物の札	`.gitignore`（Git に入れないファイル）	Git
鍵の置き場所のメモ	`.env.example`（環境変数の見本。秘密の値は書かない）	人（コピーして `.env` にする）

> **用語:** 版の固定（はんのこてい：使う道具や部品の版を、1つに決めて書いておくこと。英語では pin（ピン留め）とも言う）

> **用語:** 依存クレート（いぞんクレート：自分のプログラムが使う、ほかの人が作ったクレート。そのクレートが使うクレートも、間接的な依存になる）

図で理解する

6つのファイルを、読む道具ごとに並べると、次のようになります。

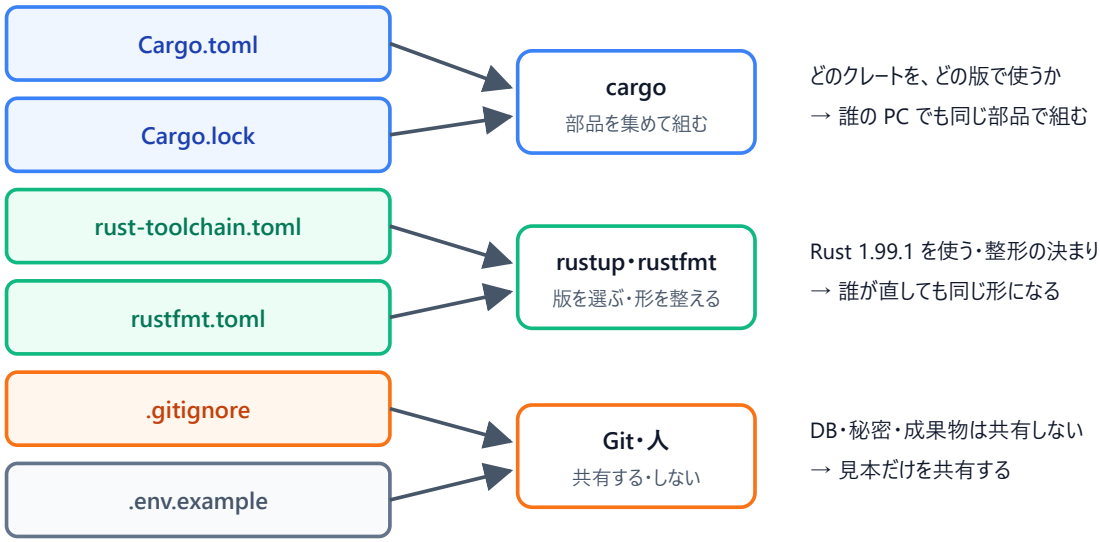


図13-3: 6つの設定ファイルと、それを読む道具。cargo が部品をそろえ、rustup と rustfmt が版と形をそろえ、Git と人が「共有しない物」を分ける

- **青の2つ**（`Cargo.toml`・`Cargo.lock`）は、cargo が読みます。使う部品と、その版を決めます。

- **緑の2つ**（`rust-toolchain.toml`・`rustfmt.toml`）は、rustup と rustfmt が読みます。Rust そのものの版と、コードの見た目を決めます。
- **橙と灰の2つ**（`.gitignore`・`.env.example`）は、Git と人のためのファイルです。共有してはいけない物（DB のファイル・秘密の値）を、共有する物から分けます。

`Cargo.toml` に書いたクレートは 10 個だけですが、それぞれが別のクレートを使っているので、実際に必要になる部品はずっと多くなります。



Cargo.toml は人が書く「材料表」、Cargo.lock は cargo が書く「レシート」

図13-4: 直接使う 10 個のクレートから、cargo が版を選んで 361 個の一覧（Cargo.lock）を作り、この PC（Windows ARM64）でのビルドには、そのうち 289 個を使う

- 361 個は、`Cargo.lock` の `[[package]]` の数です（RustNote 自身を含みます）。289 個は、この PC で `cargo build` したときに表示された `Compiling` の行の数です。この PC では使わない、ほかの OS 用のクレート（Linux や macOS でだけ使う物など）と、テストでだけ使うクレートは、`cargo build` ではコンパイルされません。
- 数はこれから先の版で変わりますが、「直接書いたのは少しでも、実際の部品はその何十倍にもなる」ことは変わりません。だから、版を人の記憶に頼らず、ファイルで固定しておくのです。

まず動かす最小コード

この単元の「コード」は、設定ファイルそのものです。まず全文を読み、次に、その設定で RustNote がビルドでき、確かめの3つのコマンドが通ることを確かめます。

例1-1: Cargo.toml の全文

> **▼ このファイルがやること（先に日本語で）**：パッケージの名前と版、使う Rust の最低の版（1.99）、使うクレート（9個）とテストでだけ使うクレート（1個）を、版を `=` で1つに固定して書きます。tokio と serde には、使う機能（features）も書きます。

```toml
[package]
name = "rustnote"
version = "0.1.0"
edition = "2024"
# Topcoat 0.11 は Rust 1.99 以上が必要（動作確認は 1.99.1）
rust-version = "1.99"
description = "RustNote: Topcoat + libSQL で作るログイン付きメモアプリ（教材のサンプル）"
publish = false
default-run = "rustnote"

# 教材と同じ結果になるよう、直接使うクレートの版は `=` で固定している。
# 間接的に使われるクレートの版は Cargo.lock で固定される。
[dependencies]
# アプリの入口（main）でエラーに文脈を付けて表示する
anyhow = "=1.0.104"
# パスワードのハッシュ化（Argon2id）
argon2 = "=0.6.0"
# trait に async fn を書き、dyn（Arc<dyn NoteRepository + Send + Sync>）で使えるようにする
async-trait = "=0.1.92"
# .env ファイルを環境変数として読み込む（開発用）
dotenvy = "=0.15.7"
# Turso / libSQL のクライアント（ローカルファイルとリモートの両方）
libsql = "=0.10.0"
# フォームや JSON を構造体に変換する
serde = { version = "=1.0.229", features = ["derive"] }
# エラー型を短く書く
thiserror = "=2.0.21"
# 非同期ランタイム
tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macros"] }
# Web フレームワーク
topcoat = "=0.11.0"

[dev-dependencies]
# テストごとに使い捨ての一時フォルダ（DB ファイル置き場）を作る
tempfile = "=3.27.0"
```

▼ コードを1つずつ分解して解説

解説1: [package] は、このパッケージの名札

- `name = "rustnote"` は、パッケージの名前です。ライブラリを外から使うときの `rustnote::domain` の `rustnote` は、この名前です（第9巻 UNIT 04）。
- `edition = "2024"` は、Rust の書き方の世代です。同じ Rust 1.99.1 でも、edition が違うと、使える書き方や警告が少し変わります。

- `rust-version = "1.99"` は、「このパッケージをビルドするには、Rust 1.99 以上が必要」という宣言です。Topcoat 0.11 が 1.99 以上を求めているので、それに合わせています。これより古い Rust でビルドしようとすると、コンパイルを始める前に止まります（エラー1）。
- `publish = false` は、`cargo publish` で crates.io（クレートの公開場所）に**うっかり公開しない**ための印です。RustNote は教材のアプリなので、公開しません。
- `default-run = "rustnote"` は、`cargo run` で動かす実行ファイルを決めます。RustNote の実行ファイルは `src/main.rs` の1つだけなので、無くても同じように動きます。あとで `src/bin/` に道具の実行ファイルを足したときも、`cargo run` が RustNote 本体を選ぶように、先に書いてあります。

解説2: "=1.54.1" は「この版だけ」

- `anyhow = "=1.0.104"` のように、版の前に `=` を付けると、**その版だけ**を使います。`=` を付けずに `"1.0.104"` と書くと、「1.0.104 以上、2.0.0 未満」の意味になり、cargo はその範囲で新しい版を選んでよくなります（例1-3で確かめます）。
- RustNote は、教材の出力と同じ結果になるように、直接使うクレートを全部 `=` で固定しています。`Cargo.toml` の上のコメント「教材と同じ結果になるよう、直接使うクレートの版は `=` で固定している」が、その理由です。
- 一方で、間接的に使われるクレートの版は、`Cargo.toml` には書きません。それを固定するのが `Cargo.lock` です（解説4）。

解説3: features は、クレートの機能のスイッチ

- `tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macros"] }` は、tokio の機能のうち、複数のスレッドで動くランタイム（`rt-multi-thread`）と、`#[tokio::main]` や `#[tokio::test]` を書くためのマクロ（`macros`）を使う、という意味です（第4巻 UNIT 07、第5巻 UNIT 02）。
- `serde = { version = "=1.0.229", features = ["derive"] }` の `derive` は、`#[derive(Deserialize)]` と書いてフォームの値を構造体に変える機能です。
- 機能を細かく選べるのは、使わない機能のぶんだけ、コンパイルの時間と、できあがるプログラムの大きさを減らせるからです。ただし、features には「依存のつながり全体で足し算される」という性質があります（例1-4）。

解説4: [dev-dependencies] は、テストのときだけ

- `tempfile` は、テストごとに使い捨ての一時フォルダを作るクレートです。`[dev-dependencies]` に書いたクレートは、`tests/`・`examples/`・`#[cfg(test)]` のテストからだけ使えます。本番の実行ファイルには入りません（問1で確かめます）。

例1-2: rust-toolchain.toml と rustfmt.toml の全文

> **▼ このファイルがやること（先に日本語で）：** `rust-toolchain.toml` は、このフォルダの中で `cargo` や `rustc` を実行したときに使う Rust の版を 1.99.1 に決め、rustfmt と clippy も一緒に入れるよう指定します。`rustfmt.toml` は、`cargo fmt` が edition 2024 の決まりで整形するよう指定します。

```
# このフォルダの中で cargo を実行すると、Rust 1.99.1 が使われる
#  （既定のツールチェーンは変更しない）
[toolchain]
channel = "1.99.1"
components = ["rustfmt", "clippy"]
```

```
# rustfmt（cargo fmt / topcoat fmt --rustfmt）の設定
edition = "2024"
```

`Cargo.toml` の `rust-version = "1.99"` は「1.99 以上なら何でもよい」、`rust-toolchain.toml` の `channel = "1.99.1"` は「このフォルダでは 1.99.1 を使う」です。最低の版と、実際に使う版を、別々のファイルで決めています。

例1-3: 決めた版でビルドし、3つの確かめを通す

> **▼ このコマンドがやること（先に日本語で）**：コピーしたフォルダで、`Cargo.lock` を書き換えないようにして（`--locked`）、RustNote をビルドします。続いて、整形の確かめ・Clippy・テストの3つを、`ci.yml` と同じ形で実行します。

```
# このフォルダで使われる Rust の版を確かめる
rustup show active-toolchain
# Cargo.lock のとおりの版でビルドする
cargo build --locked
# 確かめの3つ（ci.yml と同じ）
cargo fmt --check
cargo clippy --all-targets --locked -- -D warnings
cargo test --locked
```

▼ 実行結果（`cargo build --locked` は `Compiling` の行が 289 行続くので、最後の3行だけを載せています）

```
1.99.1-aarch64-pc-windows-msvc (overridden by '...\rustnote\rust-toolchain.toml')
```

```
   Compiling libsql v0.10.0
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 54.58s
```

`cargo fmt --check` は、整形が必要な所が無ければ、何も表示しません。`cargo clippy` の最後の3行と、`cargo test` の区切りと結果の行です。

```
    Checking libsql v0.10.0
    Checking rustnote v0.1.0 (...\rustnote)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 32.68s
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 13.22s
     Running unittests src\lib.rs (target\debug\deps\rustnote-c39f4ee352b39312.exe)
test result: ok. 19 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished
in 0.00s
     Running unittests src\main.rs (target\debug\deps\rustnote-009228cf007e52ec.exe)
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
     Running tests\http.rs (target\debug\deps\http-afc265ac134dcb62.exe)
test result: ok. 14 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished
in 2.32s
     Running tests\repository.rs (target\debug\deps\repository-74d08b56e783b6bc.exe)
test result: ok. 8 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.24s
   Doc-tests rustnote
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

▼ コードを1つずつ分解して解説

解説1: overridden by は「フォルダの設定で上書きした」

- `rustup show active-toolchain` の `(overridden by '...\rustnote\rust-toolchain.toml')` は、「PC の既定の版ではなく、このファイルで決めた版を使っています」という意味です。この PC の既定の版は 1.97.1 ですが、RustNote のフォルダでは 1.99.1 が使われています。
- だから、PC ごとに既定の版が違っても、RustNote のフォルダの中では、全員が同じ 1.99.1 でビルドします。

解説2: --locked は「Cargo.lock を書き換えるな」

- `cargo build` は、`Cargo.toml` と `Cargo.lock` が食い違っていると、`Cargo.lock` を自動で書き直してからビルドします。`--locked` を付けると、書き直す必要があるときは、ビルドせずにエラーで止まります（エラー3）。
- 「レシートと違う物を、黙って買ってこない」ための指定です。RustNote の `Dockerfile`（`cargo build --release --locked`）と `ci.yml` も、`--locked` を付けています。

解説3: 3つの確かめは、全部「何も言われない」が合格

- `cargo fmt --check` は、何も表示しないのが合格です。`cargo clippy ... -- -D warnings` は、警告が1つでもあればエラーにします。`Finished` で終われば合格です。`cargo test --locked` は、全部の `test result:` が `ok.` なら合格です。
- RustNote のテストは、ユニットテスト 19 件、`tests/http.rs` の 14 件、`tests/repository.rs` の 8 件、合わせて 41 件です。

もう一歩: 実務に近い形

例1-4: 直接の依存を一覧にする

> **▼ このコマンドがやること（先に日本語で）:** `cargo tree` は、依存のつながりを木の形で表示します。`--depth 1` で、RustNote が直接使うクレートだけに絞ります。

```
cargo tree --locked --depth 1
```

▼ 実行結果

```
rustnote v0.1.0 (...\rustnote)
├── anyhow v1.0.104
├── argon2 v0.6.0
├── async-trait v0.1.92 (proc-macro)
├── dotenvy v0.15.7
├── libsql v0.10.0
├── serde v1.0.229
├── thiserror v2.0.21
├── tokio v1.54.1
└── topcoat v0.11.0
[dev-dependencies]
└── tempfile v3.27.0
```

- `Cargo.toml` に書いた 10 個が、`Cargo.lock` で選ばれた版で並んでいます。`[dev-dependencies]` の下が、テストでだけ使う `tempfile` です。
- `async-trait` の `(proc-macro)` は、「コンパイルのときにコードを書き足すマクロのクレート」という印です。`#[async_trait]` の印を付けた trait を、dyn で使える形に書き換えます（第9巻 UNIT 04）。

例1-5: "=1.53.3" と "1.53.3" の違いを確かめる

> **▼ このコマンドがやること（先に日本語で）:** `cargo update -p tokio --dry-run` は、「tokio を新しくしたら、どの版になるか」を、`Cargo.lock` を書き換えずに表示します（`--dry-run` は「試しに動かすだけ」）。ただし、今の RustNote の tokio（1.54.1）は、いちばん新しい版なので、このままでは違いが見えません。そこで、練習用のコピーで、tokio を1つ前の 1.53.3 に固定し直してから試します。確かめたら、`Cargo.toml` と `Cargo.lock` を教材のものに戻します。

```
# Cargo.toml の tokio の行を version = "=1.53.3" に変えてから、Cargo.lock も 1.53.3 にする
cargo update -p tokio --precise 1.53.3
```

```
    Updating crates.io index
 Downgrading tokio v1.54.1 -> v1.53.3
note: pass `--verbose` to see 1 unchanged dependencies behind latest
```

```
# =1.53.3 のまま
cargo update -p tokio --dry-run
```

```
    Updating crates.io index
     Locking 0 packages to highest Rust 1.99 compatible versions
note: pass `--verbose` to see 2 unchanged dependencies behind latest
warning: not updating lockfile due to dry run
```

`Cargo.toml` の tokio の行を、`version = "=1.53.3"` から `version = "1.53.3"`（`=` を消しただけ）に変えて、同じコマンドを実行します。

```
    Updating crates.io index
     Locking 1 package to highest Rust 1.99 compatible version
    Updating tokio v1.53.3 -> v1.54.1
note: pass `--verbose` to see 1 unchanged dependencies behind latest
warning: not updating lockfile due to dry run
```

- `=` があるときは、`Locking 0 packages`（変える版は 0 個）でした。`=1.53.3` の範囲には、1.53.3 しか無いからです。
- `=` を消すと、`Updating tokio v1.53.3 -> v1.54.1` と表示されました。`"1.53.3"` は「1.53.3 以上、2.0.0 未満」の意味なので、この範囲でいちばん新しい 1.54.1 を選べるのです。
- `to highest Rust 1.99 compatible version` は、「Rust 1.99 で使える版の中で、いちばん新しい版」という意味です。cargo は、`Cargo.toml` の `rust-version` を見て、それより新しい Rust を求めるクレートの版を、選ばないようにしています。

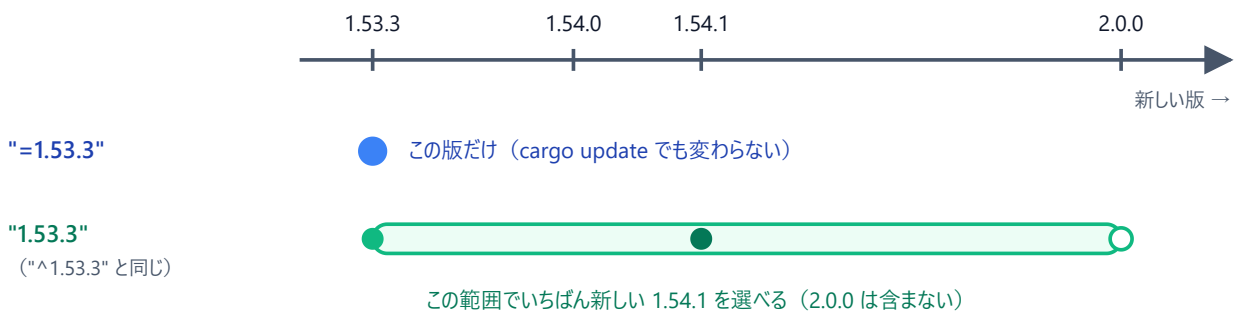


図13-5: 版の書き方と、選べる版の範囲。= を付けるとその版だけ、付けないと「上の桁が同じ範囲」の新しい版まで選べる

- `2 unchanged dependencies behind latest` は、「もっと新しい版が出ているのに、今の決まりでは選べないクレートが 2 個ある」という意味です。`=` を消すと 1 個に減りました。tokio が、新しい版を選べるようになったからです。`=` で固定すると、新しい版の改善も、自分で版を上げるまで入りません。固定は「同じ結果」の代わりに、「版を上げる作業を自分で決める」ことでもあります。

> **注意:** `--dry-run` を付け忘れると、`Cargo.lock` が本当に書き換わります。この例を試すときは、必ず `--dry-run` を付け、終わったら `Cargo.toml` の `=` を元に戻してください。

例1-6: features は、依存のつながり全体で足し算される

> **▼ このコマンドがやること（先に日本語で）：** `cargo tree -p tokio --depth 0 -f "{p} [{f}]"` は、tokio だけを表示し、`{f}` の所に、**実際に有効になっている機能**を並べます。

```
cargo tree --locked -p tokio --depth 0 -e normal -f "{p} [{f}]"
```

▼ 実行結果

```
tokio v1.54.1 [bytes,default,fs,full,io-std,io-
util,libc,macros,mio,net,parking_lot,process,rt,rt-multi-thread,signal,signal-hook-
registry,socket2,sync,time,tokio-macros,windows-sys]
```

- RustNote の `Cargo.toml` に書いた機能は `rt-multi-thread` と `macros` の2つだけなのに、`fs`・`net`・`signal`・`full` など、たくさんの機能が有効になっています。
- これは、RustNote が使うほかのクレートも、tokio の機能を求めているからです。`cargo tree -e features -i tokio` で調べると、libsql が使う `libsql_replication` が tokio の `full`（ほぼ全部の機能）を、`topcoat` が `macros`・`net`・`signal` を有効にしていることが分かります（それぞれのクレートの `Cargo.toml` にも、そう書いてあります）。cargo は、1つのビルドの中の tokio を1つにまとめ、**誰かが求めた機能を全部足したもの**でコンパイルします。
- そのため、試しに RustNote の `Cargo.toml` から `"macros"` を消しても、ビルドは通ってしまいます（`cargo build --locked` は `Finished` で終わりました）。ほかのクレートが `macros` を有効にしてくれているからです。
- それでも、RustNote の `Cargo.toml` には `macros` を書いておきます。libsql や topcoat が次の版で tokio の使い方を変えたら、RustNote の `#[tokio::main]` は突然コンパイルできなくなるからです。**自分が使う機能は、自分の `Cargo.toml` に書く**のが決まりです。だれも `macros` を有効にしていないと何が起きるかは、エラー5で確かめます。

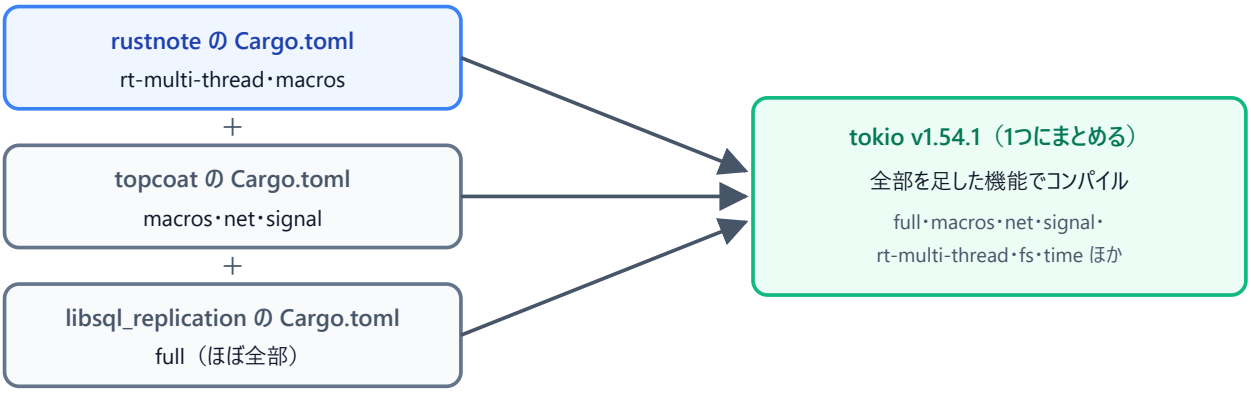


図13-6: features の足し算。1つのビルドの中の tokio は1つにまとめられ、誰かが求めた機能を全部足したものでコンパイルされる。だから自分が書き忘れた機能も、ほかのクレートのおかげで使えてしまうことがある

features とは別の話ですが、1つのビルドの中に、版の違う同じクレートが同時に入ることもあります。`Cargo.lock` の RustNote の項目を見てみましょう（抜き出しています）。

```
[[package]]
name = "rustnote"
version = "0.1.0"
dependencies = [
 "anyhow",
 "argon2",
 "async-trait",
 "dotenvy",
 "libsql",
 "serde",
 "tempfile",
 "thiserror 2.0.21",
 "tokio",
 "topcoat",
]
```

- `"thiserror 2.0.21"` だけ、名前の後ろに版が付いています。`Cargo.lock` に thiserror が 1.0.69 と 2.0.21 の2つあるので、どちらかを区別するためです。

```
cargo tree --locked -i thiserror@1.0.69 --edges normal
```

```
thiserror v1.0.69
├── libsql v0.10.0
│   └── rustnote v0.1.0 (...\rustnote)
└── libsql_replication v0.10.0
    └── libsql v0.10.0 (*)
```

- `-i`（invert：逆向き）は、「このクレートを使っているのは誰か」をたどります。thiserror 1.0.69 は libsql が使い、RustNote 自身は 2.0.21 を使っていました。1.x と 2.x のように、上の桁が違う版は「互換性のない別物」とみなされるので、1つのビルドの中に両方が入ってもかまわないのです。

例1-7: .gitignore と .env.example の全文

> **▼ このファイルがやること（先に日本語で）：** `.gitignore` は、Git に入れないファイルの名前の決まりを書きます。`.env.example` は、RustNote が読む環境変数の名前と、開発用の値の見本を書きます。秘密の値（Turso のトークン）は、空のままにしてあります。

```
# ビルドの成果物（cargo build / topcoat asset bundle が作る）
/target/

# ローカルの SQLite / libSQL データベース（WAL などの付属ファイルも含む）
*.db
*.db-shm
*.db-wal
*.db-journal

# 秘密の値を書いた環境変数ファイル（.env.example だけを共有する）
.env

# OS やエディタが作るファイル
.DS_Store
Thumbs.db
```

```
# RustNote の環境変数の見本。
# このファイルを .env という名前でコピーし、値を書き換えて使う（.env は Git に入れない）。

# サーバーが待ち受けるアドレスとポート（topcoat::start が読む）
# 開発: 127.0.0.1:3000 / Fly.io: 0.0.0.0:8080
HOST=127.0.0.1
PORT=3000

# データベース
# 開発: ローカルファイルのパス（無ければ自動で作られる）
DATABASE_URL=rustnote.db
# 本番（Turso）の例:
# DATABASE_URL=libsql://rustnote-<あなたの組織名>.turso.io
# DATABASE_AUTH_TOKEN=<turso db tokens create で作ったトークン>
DATABASE_AUTH_TOKEN=
```

- `.gitignore` の `*.db` は「名前が `.db` で終わるファイル全部」です。開発で作った DB には、自分で試しに登録したメールアドレスなどが入るので、Git に入れません。`*.db-shm`・`*.db-wal`・`*.db-journal` は、SQLite が DB の横に作る作業用のファイルです。
- `.env` は Git に入れず、`.env.example` だけを共有します。`.env.example` を `.env` という名前でコピーし、自分の値を書き込んで使います。RustNote の `main.rs` は、起動するときに `dotenvy::dotenv().ok()` で `.env` を読みます（`.env` が無ければ何もしません。UNIT 03（準備中））。
- `Cargo.lock` は `.gitignore` に**書いていません**。アプリ（実行ファイルを作るパッケージ）では、`Cargo.lock` を Git に入れて共有するのが、Cargo の公式の考え方です。共有しないと、PC ごとに違う版の部品でビルドすることになるからです。

よくあるエラーと読み方

この節のエラーは、どれも練習用のコピーの設定ファイルを1か所だけ書き換えて出したものです。確かめ終わったら、そのファイルを必ず元に戻してください。

エラー1: Rust の版が古い（rust-version）

`rust-toolchain.toml` が無いフォルダや、`cargo +stable build` のように版を指定して、Rust 1.97.1 でビルドしようとした場合です（この PC の既定の版 `stable` は 1.97.1 です）。

```
cargo +stable check --locked
```

（`requires rustc 1.99` の行が 35 行続くので、途中を省いています）

```
error: rustc 1.97.1 is not supported by the following packages:
  rustnote@0.1.0 requires rustc 1.99
  rustnote@0.1.0 requires rustc 1.99
  topcoat@0.11.0 requires rustc 1.99
  topcoat-asset@0.11.0 requires rustc 1.99
  …（途中を省略）…
  topcoat-view-grammar@0.11.0 requires rustc 1.99
  topcoat-view-macro@0.11.0 requires rustc 1.99
```

- **何が起きたか:**「rustc 1.97.1 は、次のパッケージに対応していません」というエラーです。`rustnote@0.1.0 requires rustc 1.99` は、RustNote の `Cargo.toml` の `rust-version = "1.99"`、`topcoat@0.11.0 requires rustc 1.99` は、Topcoat 側の同じ宣言です。同じ名前が何回か出ていますが、読むのは、パッケージの名前と `requires rustc 1.99` の部分だけで十分です。
- このエラーは、**コンパイルを始める前に**出ます。古い Rust で途中まで長いコンパイルをして、分かりにくいエラーで止まるより、ずっと親切です。`rust-version` を書いておく理由がこれです。

- **直し方**: RustNote のフォルダでは、`+stable` を付けずに `cargo check --locked` とします。`rust-toolchain.toml` があるので 1.99.1 が使われます。1.99.1 がまだ入っていない PC では、`rustup toolchain install 1.99.1 --profile minimal --component rustfmt,clippy` で入れます（既定の版は変わりません）。

> **注意**: `cargo +1.97.1 check` のように、PC に入っていない版の名前を指定すると、この PC の rustup は、その版を**自動でダウンロードして入れました**（`warn: the missing active toolchain ... has been auto-installed` と表示されます）。版を試すときは、`rustup toolchain list` で入っている版を確かめてから指定しましょう。入れてしまった版は、`rustup toolchain uninstall 版の名前` で消せます。

エラー2: 機能の名前を打ち間違えた

`Cargo.toml` の tokio の `"macros"` を、`"macro"`（`s` が無い）と打ち間違えた場合です。

```
tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macro"] }
```

```
cargo build --locked
```

```
error: failed to select a version for `tokio`.
    ... required by package `rustnote v0.1.0 (...\rustnote)`
versions that meet the requirements `=1.54.1` (locked to 1.54.1) are: 1.54.1

package `rustnote` depends on `tokio` with feature `macro` but `tokio` does not have
that feature.
help: there is a feature `macros` with a similar name


failed to select a version for `tokio` which could resolve this conflict
```

- **何が起きたか**:「tokio の版を選べませんでした」というエラーです。その下の `package rustnote depends on tokio with feature macro but tokio does not have that feature.`（RustNote は tokio の `macro` という機能を求めていますが、tokio にその機能はありません）が本当の理由です。
- `versions that meet the requirements =1.54.1 (locked to 1.54.1) are: 1.54.1` は、「条件に合う版は 1.54.1 だけで、その版には `macro` という機能が無いので、選べる版がもう無い」という意味です。
- **直し方**: `help: there is a feature macros with a similar name`（似た名前の `macros` という機能があります）のとおり、`"macros"` に直します。機能の名前は、クレートの docs.rs のページの「Feature flags」で確かめられます。

エラー3: --locked なのに、Cargo.lock を書き換える必要がある

`Cargo.toml` の dotenvy の版を、`"=0.15.7"` から `"=0.15.6"` に変えて、`--locked` でビルドした場合です。

```
cargo build --locked
```

```
    Updating crates.io index
error: cannot update the lock file ...\rustnote\Cargo.lock because --locked was passed
to prevent this
help: to generate the lock file without accessing the network, remove the --locked flag
and use --offline instead.
```

- **何が起きたか:** 「`--locked` が指定されたので、Cargo.lock を書き換えられません」というエラーです。`Cargo.toml` は 0.15.6 を求めているのに、`Cargo.lock` には 0.15.7 が書いてあるので、ビルドするには `Cargo.lock` を書き直す必要がありました。
- **これはエラーで正しい:** CI や Docker のビルドで、`Cargo.lock` と違う版が黙って使われるのを止めるために、わざと `--locked` を付けています。
- **直し方:** 版を本当に変えたいときは、`--locked` を付けずに `cargo build`（または `cargo update -p dotenvy`）を実行して `Cargo.lock` を更新し、`Cargo.toml` と `Cargo.lock` を**一緒に**コミットします。この練習では、`Cargo.toml` を `"=0.15.7"` に戻します。
- `help:` の `--offline`（ネットにつながずに作る）の案内は、ネットを使わずに `Cargo.lock` を作り直したいときの方法です。今回は版を戻せばよいので、使いません。

エラー4: Cargo.lock の指紋（checksum）が違う

`Cargo.lock` の dotenvy の `checksum` の、先頭の1文字だけを書き換えた場合です。

```
[[package]]
name = "dotenvy"
version = "0.15.7"
source = "registry+https://github.com/rust-lang/crates.io-index"
checksum = "0aaf95b3e5c8f23aa320147307562d361db0ae0d5124234 0f558153b4eb2439b"
```

```
cargo build --locked
```

```
error: checksum for `dotenvy v0.15.7` changed between lock files

this could be indicative of a few possible errors:

    * the lock file is corrupt
    * a replacement source in use (e.g., a mirror) returned a different checksum
    * the source itself may be corrupt in one way or another

unable to verify that `dotenvy v0.15.7` is the same as when the lockfile was generated
```

- **何が起きたか**: 「dotenvy 0.15.7 の checksum が、ロックファイルの間で変わりました」というエラーです。`checksum` は、crates.io から取ってきたクレートの中身から計算した**指紋**（SHA-256）です。本物の指紋は `1aaf95b3…` で始まるのに、`Cargo.lock` には `0aaf95b3…` と書いてあったので、cargo は「同じ物か確かめられない」と止まりました。
- その下の3つの点は、考えられる原因です。ロックファイルが壊れている、ミラー（代わりの取得先）が違う中身を返した、取得先そのものが壊れている、のどれかです。
- **これはエラーで正しい**: 途中で中身をすり替えられたクレートを、気づかずにビルドしてしまう事故を防いでいます。`Cargo.lock` を Git で共有すると、この指紋も全員で共有されます。
- **直し方**: `Cargo.lock` は手で書き換えるファイルではありません（先頭に `It is not intended for manual editing.` と書いてあります）。教材のフォルダから `Cargo.lock` をコピーし直します。

エラー5: 誰も macros を有効にしていない（E0433・E0752）

例1-6で、RustNote では `"macros"` を消してもビルドが通ることを見ました。では、ほかのクレートが助けてくれない場合はどうなるでしょう。tokio だけを使う小さなプロジェクト（`feat_demo`）で試します。

```
[package]
name = "feat_demo"
version = "0.1.0"
edition = "2024"

[dependencies]
# RustNote と同じ版の tokio を、macros なしで使う
tokio = { version = "=1.54.1", features = ["rt-multi-thread"] }
```

```
// #[tokio::main] は、tokio の macros の機能が有効なときだけ使える
#[tokio::main]
async fn main() {
    println!("tokio のランタイムの中で動いています");
}
```

```
cargo build
```

```
   Compiling tokio v1.54.1
   Compiling feat_demo v0.1.0 (...\feat_demo)
error[E0433]: cannot find `main` in `tokio`
 --> src\main.rs:2:10
  |
2 | #[tokio::main]
  |          ^^^^ could not find `main` in `tokio`

error[E0752]: `main` function is not allowed to be `async`
 --> src\main.rs:3:1
  |
3 | async fn main() {
  | ^^^^^^^^^^^^^^^ `main` function is not allowed to be `async`

Some errors have detailed explanations: E0433, E0752.
For more information about an error, try `rustc --explain E0433`.
error: could not compile `feat_demo` (bin "feat_demo") due to 2 previous errors
```

- **何が起きたか:** 1つ目の `error[E0433]: cannot find main in tokio` は、「tokio の中に `main` が見つかりません」です。`#[tokio::main]` は、`macros` の機能を有効にしたときだけ tokio の中に現れるので、機能が無いと「無い物を使った」エラーになります。
- 2つ目の `error[E0752]: main function is not allowed to be async` は、「`main` 関数を `async` にはできません」です。本当は `#[tokio::main]` が `async fn main` を普通の `fn main` に書き換えてくれるはずでしたが、1つ目のエラーでそれが働かなかったので、続けてこのエラーが出ました。**最初のエラーから直す**と、2つ目は一緒に消えます。
- **直し方:** `Cargo.toml` の features に `"macros"` を足します。

```toml
[package]
name = "feat_demo"
version = "0.1.0"
edition = "2024"

[dependencies]
# macros を足すと、#[tokio::main] が使える
tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macros"] }
```

```
cargo run -q
```

```
tokio のランタイムの中で動いています
```

RustNote ではここで使う

この単元のファイルは、RustNote のほかのファイルから、次のように使われています。

使う場所	書いてあること	つながるファイル
`Dockerfile`	`FROM rust:1.99.1-bookworm AS builder`	`rust-toolchain.toml` と同じ 1.99.1
`Dockerfile`	`RUN cargo install topcoat-cli --version 0.11.0 --locked`	`Cargo.toml` の topcoat と同じ 0.11.0
`Dockerfile`	`RUN cargo build --release --locked`	`Cargo.lock`（`--locked`）
`ci.yml`	`toolchain: 1.99.1` と `components: rustfmt, clippy`	`rust-toolchain.toml`
`ci.yml`	`cargo clippy --all-targets --locked -- -D warnings`・`cargo test --locked`	`Cargo.lock`
`.dockerignore`	`target/`・`*.db`・`.env` を送らない	`.gitignore` と同じ考え方
`src/main.rs`	`dotenvy::dotenv().ok();`	`.env.example` をコピーした `.env`

- Rust の版（1.99.1）は、`rust-toolchain.toml`・`Dockerfile`・`ci.yml` の3か所に書いてあります。版を上げるときは、この3か所を**一緒に**変えます。1か所だけ変えると、「自分の PC では通るのに、CI では通らない」という食い違いが起きます。
- topcoat の版（0.11.0）も、`Cargo.toml` のクレートと、`Dockerfile` の CLI の2か所です。クレートと CLI は同じ Topcoat の一部なので、版を上げるときは両方を一緒に上げます。

練習問題

問1: `tempfile` は `[dev-dependencies]` に書いてあります。もし `src/lib.rs` の中で `tempfile::tempdir()` を使う関数を書いたら、どうなるでしょうか。予想してから、練習用のコピーで確かめてください（確かめたら元に戻します）。また、なぜ `tempfile` を `[dependencies]` ではなく `[dev-dependencies]` に書くのかを答えてください。

問2: ARM64 版の Windows で RustNote をビルドするには、LLVM（clang）が必要でした（第9巻の巻頭）。これは、暗号のクレート `ring` を C のコンパイラでビルドするためです。では、RustNote の `Cargo.toml` には `ring` が無いのに、なぜ `ring` がビルドされるのでしょうか。`cargo tree` で、`ring` を使っているクレートをたどってください。

問3: 練習用のコピーで `git init` し、`.env.example` を `.env` にコピーし、`rustnote.db` という名前の空でないファイルを作ります。そのうえで、次の2つを確かめてください。

1. `git check-ignore -v` で、`.env`・`rustnote.db`・`rustnote.db-wal`・`target/debug/rustnote.exe`・`.env.example`・`Cargo.lock` のうち、どれが Git から外されるか。外されるものは、`.gitignore` の何行目の決まりによるか。
2. `git status --short` に、`.env` と `rustnote.db` が出てこないこと。

解答と解説

問1の解答:

予想: コンパイルエラーになる。`[dev-dependencies]` のクレートは、ライブラリの本体（`src/`）からは見えないからです。`src/lib.rs` の最後に、次の関数を足して確かめます。

```
/// 試し: dev-dependencies のクレートを src/ で使う
pub fn scratch_dir() -> tempfile::TempDir {
    tempfile::tempdir().expect("一時フォルダを作れません")
}
```

```
cargo build --locked
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0433]: cannot find module or crate `tempfile` in this scope
  --> src\lib.rs:30:5
   |
30 |     tempfile::tempdir().expect("一時フォルダを作れません")
   |     ^^^^^^^^ use of unresolved module or unlinked crate `tempfile`
   |
   = help: if you wanted to use a crate named `tempfile`, use `cargo add tempfile` to
add it to your `Cargo.toml`

error[E0433]: cannot find module or crate `tempfile` in this scope
  --> src\lib.rs:29:25
   |
29 | pub fn scratch_dir() -> tempfile::TempDir {
   |                         ^^^^^^^^ use of unresolved module or unlinked crate
`tempfile`
   |
   = help: if you wanted to use a crate named `tempfile`, use `cargo add tempfile` to
add it to your `Cargo.toml`

For more information about this error, try `rustc --explain E0433`.
error: could not compile `rustnote` (lib) due to 2 previous errors
```

- `cannot find module or crate tempfile in this scope`（この場所では `tempfile` というモジュールもクレートも見つかりません）と、使った2か所でエラーになりました。`help:` は「`cargo add tempfile` で `Cargo.toml` に足してください」と案内していますが、テストのための道具を本体の依存にするのは、正しい直し方ではありません。
- `[dev-dependencies]` に書くのは、本番の実行ファイルに**入れる必要が無い**からです。入れないぶん、本番のビルドが速くなり、できあがる実行ファイルも小さくなり、本番で動くコードが使うクレートの数（気をつけて見ておくべき範囲）も減ります。
- 確かめたら、足した関数を消して、`src/lib.rs` を元に戻します。

問2の解答:

```
cargo tree --locked -i ring --edges normal
```

```
ring v0.17.14
├── rustls v0.22.4
│   ├── hyper-rustls v0.25.0
│   │   └── libsql v0.10.0
│   │       └── rustnote v0.1.0 (...\rustnote)
│   └── tokio-rustls v0.25.0
│       └── hyper-rustls v0.25.0 (*)
└── rustls-webpki v0.102.8
    └── rustls v0.22.4 (*)
```

- 下から読むと、`rustnote → libsql → hyper-rustls → rustls → ring` とつながっています。libsql が Turso（ネットの向こうの DB）と暗号化して通信するために rustls（TLS という暗号化の仕組みを Rust で書いたクレート）を使い、rustls と、証明書を確かめる rustls-webpki が、暗号の計算に ring を使っているのです。
- `(*)` は、「この先は、上ですでに表示した」という印です。同じつながりを何度も書かないように省いています。
- RustNote の `Cargo.toml` に書いていないクレートでも、ビルドに必要な道具（ここでは clang）を求めることがあります。ビルドが止まったら、エラーに出てきたクレートの名前で `cargo tree -i` を調べると、「誰のために必要なのか」が分かります。

問3の解答:

```
git init
Copy-Item .env.example .env
Set-Content rustnote.db "x"
git check-ignore -v .env rustnote.db rustnote.db-wal target/debug/rustnote.exe
.env.example Cargo.lock
```

```
.gitignore:11:.env   .env
.gitignore:5:*.db    rustnote.db
.gitignore:7:*.db-wal    rustnote.db-wal
.gitignore:2:/target/    target/debug/rustnote.exe
```

```
git status --short
```

```
?? .dockerignore
?? .env.example
?? .gitignore
?? Cargo.lock
?? Cargo.toml
?? Dockerfile
?? README.md
?? assets/
?? ci.yml
?? fly.toml
?? migrations/
?? rust-toolchain.toml
?? rustfmt.toml
?? src/
?? tests/
```

1. 外されるのは `.env`（11 行目の `.env`）、`rustnote.db`（5 行目の `*.db`）、`rustnote.db-wal`（7 行目の `*.db-wal`）、`target/debug/rustnote.exe`（2 行目の `/target/`）の4つです。`-v`（verbose）を付けると、「どのファイルの何行目の、どの決まりに当たったか」が、`.gitignore:11:.env` の形で表示されます。`.env.example` と `Cargo.lock` は表示されません。外されない（Git で共有する）からです。
2. `git status --short` の `??`（まだ Git に入れていないファイル）の一覧に、`.env` と `rustnote.db` はありません。フォルダの中には確かにあるのに、Git からは見えていないのです。`.gitignore` の決まりの、いちばんの目的は、この「うっかり共有しない」ことです。

- `git check-ignore` は、ファイルが本当にあるかどうかに関係なく、名前だけで調べます。`target/debug/rustnote.exe` は、このフォルダにはまだありません（ビルドの成果物は別の場所に作っています）が、名前の決まりで判定されました。
- `/target/` の先頭の `/` は、「`.gitignore` のあるフォルダの直下の `target` だけ」という意味です。`src/target/` のような、下のほうのフォルダの `target` は外しません。

まとめ

- `Cargo.toml` は人が書く材料表、`Cargo.lock` は cargo が書くレシート（全クレートの版と指紋）。RustNote は直接使うクレートを `=` で固定し、`Cargo.lock` を Git で共有して、`--locked` で書き換えを止める。

- Rust の版は、`rust-version`（最低の版）と `rust-toolchain.toml`（このフォルダで使う版）で決め、`Dockerfile` と `ci.yml` にも同じ版を書く。features は依存のつながり全体で足し算されるので、自分が使う機能は自分の `Cargo.toml` に書く。
- 秘密の値と DB と成果物は `.gitignore` で共有から外し、環境変数は `.env.example` という見本だけを共有する。

UNIT 02: 表を作る SQL と DB の入口

この単元のゴール

- `migrations/` の3つの SQL ファイルが、どの表・索引・制約を作るかを説明できる
- `src/db.rs` の `migrate` が、まだ適用していない版だけを、1つずつトランザクションの中で適用し、`schema_migrations` に記録する流れを説明できる
- `Db::open_local` と `Db::connect` が入れている設定（WAL・外部キー・待ち時間）を、実際の値で確かめられる
- マイグレーションが途中で失敗したときに何が残るか、適用済みの SQL ファイルを書き換えたら何が起きるかを、実験で説明できる

たとえ話でつかむ

ある建物に、何年もかけて部屋を増やしていく場面を考えます。

- 工事には**番号**が付いています。「1番: 1階を建てる」「2番: 2階を足す」「3番: 倉庫を足す」。工事は、必ず番号の順に、1回ずつ行います。
- 建物には**工事の記録簿**があり、終わった工事の番号が書いてあります。新しい担当者が来ても、記録簿を見れば「3番まで終わっている。次は4番から」と分かります。
- 工事の途中で問題が起きたら、その工事で作りかけた物は**全部取り壊して**、工事の前の状態に戻します。半分だけできた部屋を残すと、次の工事ができなくなるからです。
- 終わった工事の**図面を書き換えても、建物は変わりません。**2番の図面に「窓を1つ増やす」と書き足しても、すでに建った2階の窓は増えません。窓を増やしたいなら、新しく「4番: 2階に窓を足す」という工事を足します。

RustNote の DB も、同じやり方で作ります。

建物の工事	RustNote
番号の付いた工事	**マイグレーション**（`migrations/0001_users.sql` など、番号順の SQL ファイル）
工事の記録簿	`schema_migrations` 表（適用済みの版の名前と時刻）
番号の順に、まだの工事だけをする	`db::migrate` が、未適用の版だけを古い順に適用する
途中で失敗したら、作りかけを取り壊す	1つの版を**トランザクション**の中で適用し、失敗したら `rollback`
終わった工事の図面を書き換えても変わらない	適用済みの SQL ファイルを書き換えても、今ある DB には反映されない（エラー2）

> **用語:** マイグレーション（migration：DB の表の作り（スキーマ）を変える手順を、番号順のファイルに分けて書いたもの。第7巻 UNIT 06）

> **用語:** トランザクション（transaction：いくつかの変更を「全部やる」か「全部やめる」かの1まとまりにする仕組み。第7巻 UNIT 05）

図で理解する

3つのマイグレーションと、それが作る物の対応です。

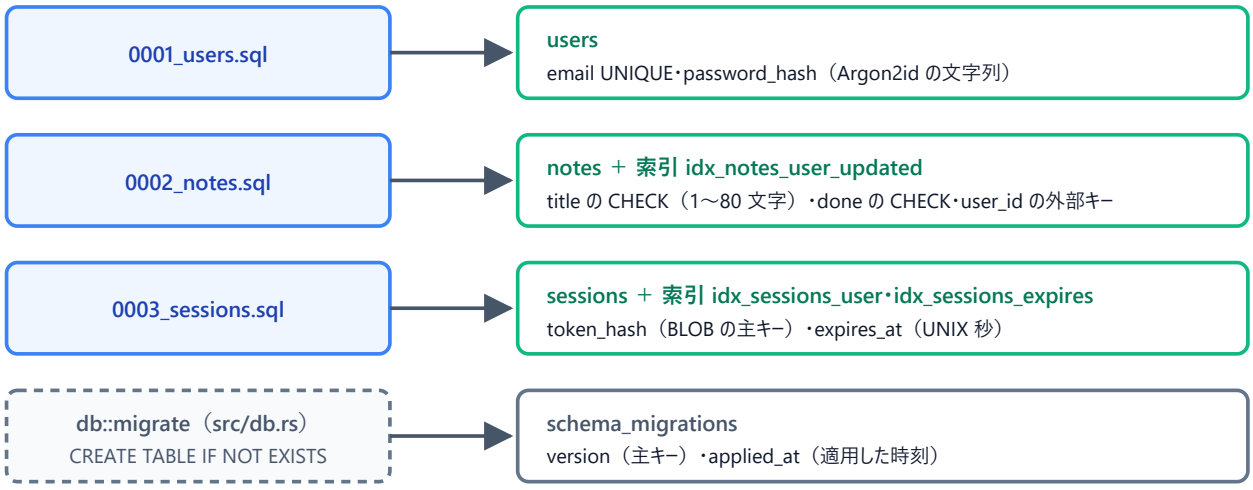


図13-7: マイグレーションのファイルと、それが作る表と索引。記録簿の schema_migrations だけは、ファイルではなく db::migrate のコードが作る

`db::migrate` は、次の順番で動きます。

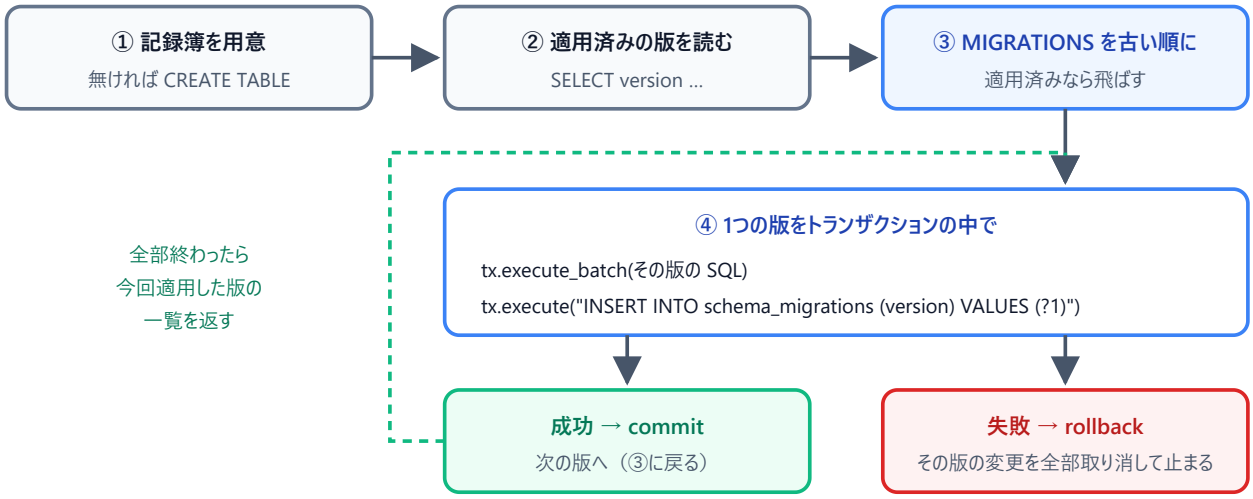


図13-8: db::migrate の流れ。適用済みの版は飛ばし、まだの版だけを1つずつトランザクションの中で適用する。SQL の実行と記録簿への書き込みは、同じトランザクションに入れる

- 大事なのは、④の中で、**SQL の実行と記録簿への書き込みを、同じトランザクションに入れている**ことです。もし SQL だけが成功して、記録が残らなかったら、次の起動で同じ SQL がもう一度流れ、「表がもうあります」のエラーになります。反対に、記録だけが残って SQL が失敗していたら、表が無いのに「適用済み」になります。同じトランザクションなら、両方が成功するか、両方とも無かったことになるかの、どちらかです。

まず動かす最小コード

例2-1: マイグレーションの SQL とコードの全文

> **▼ このファイルがやること（先に日本語で）：** `0001_users.sql` はユーザーの表、`0002_notes.sql` はノートの表と一覧を速くする索引、`0003_sessions.sql` はログイン中のセッションの表と2つの索引を作ります。どれも、決まり（`UNIQUE`・`CHECK`・外部キー）を表に書き込んでいます（第9巻 UNIT 03）。

```
-- 0001: ユーザーのテーブル
-- email は UNIQUE（同じメールアドレスで二重に登録できない）。
-- password_hash には Argon2id の PHC 文字列（$argon2id$v=19$...）を入れる。生のパスワードは保存しない。
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
```

```sql
-- 0002: ノートのテーブル
-- title は 1〜80 文字（length() は文字数を数える。Rust 側の chars().count() と同じ）。
-- done は 0（未完了）か 1（完了）。
-- version は楽観的ロック用の番号。更新のたびに 1 増える。
-- 日時は UTC の 'YYYY-MM-DD HH:MM:SS' 形式の文字列（datetime('now') の形）。
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 80),
    body TEXT NOT NULL DEFAULT '',
    done INTEGER NOT NULL DEFAULT 0 CHECK (done IN (0, 1)),
    version INTEGER NOT NULL DEFAULT 1,
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now'))
);

-- 「自分のノートを新しい順に」を速くするための索引
CREATE INDEX idx_notes_user_updated ON notes (user_id, updated_at DESC, id DESC);
```

```sql
-- 0003: ログイン中のセッションのテーブル
-- token_hash は Topcoat が発行したトークンの SHA-256（32 バイト）。生のトークンは保存しない。
-- expires_at は有効期限。1970-01-01 からの秒数（UNIX 時刻）で持ち、比較しやすくする。
CREATE TABLE sessions (
    token_hash BLOB PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    expires_at INTEGER NOT NULL,
    created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE INDEX idx_sessions_user ON sessions (user_id);
CREATE INDEX idx_sessions_expires ON sessions (expires_at);
```

▼ **このファイルがやること（先に日本語で）**： `src/db.rs` は、DB への入口の `Db` 型と、マイグレーションの `migrate` 関数を持ちます。`Db` は、設定に合わせてローカルのファイルか Turso を開き、接続のたびに必要な設定を入れます。`migrate` は、SQL ファイルを `include_str!` で実行ファイルに埋め込み、まだ適用していない版だけを適用します。

```rust
//! DB（Turso / libSQL）への接続と、マイグレーション（テーブルの作成・変更）。
//!
//! - 開発ではローカルファイル（例: rustnote.db）を使う
//! - 本番では Turso の URL（libsql://...）と認証トークンを環境変数で渡す
//!
//! 接続（Connection）は処理のたびに `Db::connect` で作る。
//! ローカルファイルなら開くだけ、リモートなら HTTP の準備だけなので軽い。

use std::path::Path;
use std::sync::Arc;

use libsql::{Builder, Connection, Database, Transaction};
use thiserror::Error;

use crate::config::Config;

/// DB への入口。中身は Arc なので、clone しても同じ DB を指す（安く複製できる）
#[derive(Clone)]
pub struct Db {
    database: Arc<Database>,
    is_local: bool,
}

impl Db {
    /// 設定に従って、ローカルファイルかリモート（Turso）の DB を開く
    pub async fn open(config: &Config) -> Result<Db, libsql::Error> {
        if config.is_remote() {
            // 秘密のトークンは、接続するこの場所でだけ取り出す
            let token = config
                .database_auth_token
                .as_ref()
                .map(|secret| secret.expose().to_string())
                .unwrap_or_default();
            let database = Builder::new_remote(config.database_url.clone(), token)
                .build()
                .await?;
            Ok(Db {
                database: Arc::new(database),
                is_local: false,
            })
        } else {
            Db::open_local(&config.database_url).await
        }
    }

    /// ローカルファイルの DB を開く（無ければ作られる）
    pub async fn open_local(path: impl AsRef<Path>) -> Result<Db, libsql::Error> {
        let database = Builder::new_local(path).build().await?;
        let db = Db {
```

```
            database: Arc::new(database),
            is_local: true,
        };
        // WAL モード: 読み込みと書き込みが同時に来ても待たされにくくなる（ファイルに記録される）
        let conn = db.connect().await?;
        conn.execute_batch("PRAGMA journal_mode = WAL;").await?;
        Ok(db)
    }

    /// 新しい接続を作る
    pub async fn connect(&self) -> Result<Connection, libsql::Error> {
        let conn = self.database.connect()?;
        if self.is_local {
            // SQLite は接続ごとに設定が必要:
            // foreign_keys … REFERENCES（外部キー）を守らせる
            // busy_timeout … ほかの接続が書き込み中なら最大 5 秒待つ
            conn.execute_batch("PRAGMA foreign_keys = ON; PRAGMA busy_timeout = 5000;")
                .await?;
        }
        Ok(conn)
    }

    /// DB に問い合わせができるか確かめる（GET /health で使う）
    pub async fn ping(&self) -> Result<(), libsql::Error> {
        let conn = self.connect().await?;
        let mut rows = conn.query("SELECT 1", ()).await?;
        rows.next().await?;
        Ok(())
    }
}

// ---------------------------------------------------------------------------
// マイグレーション
// ---------------------------------------------------------------------------

/// 1つのマイグレーション（版の名前と SQL）
pub struct Migration {
    pub version: &'static str,
    pub sql: &'static str,
}

/// アプリが持つマイグレーションの一覧（古い順）。
/// SQL ファイルは include_str! でコンパイル時に実行ファイルへ埋め込む。
pub const MIGRATIONS: &[Migration] = &[
    Migration {
        version: "0001_users",
        sql: include_str!("../migrations/0001_users.sql"),
    },
    Migration {
        version: "0002_notes",
```

```
        sql: include_str!("../migrations/0002_notes.sql"),
    },
    Migration {
        version: "0003_sessions",
        sql: include_str!("../migrations/0003_sessions.sql"),
    },
];

#[derive(Debug, Error)]
pub enum MigrateError {
    #[error("schema_migrations を準備できません")]
    Prepare(#[source] libsql::Error),
    #[error("マイグレーション {version} に失敗しました")]
    Apply {
        version: &'static str,
        #[source]
        source: libsql::Error,
    },
}

/// まだ適用していないマイグレーションだけを、1つずつトランザクションの中で適用する。
/// 戻り値は、今回新しく適用した版の名前の一覧。
pub async fn migrate(db: &Db) -> Result<Vec<&'static str>, MigrateError> {
    let conn = db.connect().await.map_err(MigrateError::Prepare)?;

    // 適用済みの版を記録する表（無ければ作る）
    conn.execute(
        "CREATE TABLE IF NOT EXISTS schema_migrations (
            version TEXT PRIMARY KEY,
            applied_at TEXT NOT NULL DEFAULT (datetime('now'))
        )",
        (),
    )
    .await
    .map_err(MigrateError::Prepare)?;

    let applied = applied_versions(&conn)
        .await
        .map_err(MigrateError::Prepare)?;

    let mut newly_applied = Vec::new();
    for migration in MIGRATIONS {
        if applied.iter().any(|v| v == migration.version) {
            continue;
        }
        let apply_error = |source| MigrateError::Apply {
            version: migration.version,
            source,
        };
        let tx = conn.transaction().await.map_err(apply_error)?;
```

```
        match apply(&tx, migration).await {
            // 全部成功したときだけ確定する
            Ok(()) => tx.commit().await.map_err(apply_error)?,
            // 途中で失敗したら、その版の変更をすべて取り消す
            Err(source) => {
                tx.rollback().await.map_err(apply_error)?;
                return Err(apply_error(source));
            }
        }
        newly_applied.push(migration.version);
    }
    Ok(newly_applied)
}

/// schema_migrations に記録された版の名前を読む
async fn applied_versions(conn: &Connection) -> Result<Vec<String>, libsql::Error> {
    let mut rows = conn
        .query("SELECT version FROM schema_migrations ORDER BY version", ())
        .await?;
    let mut versions = Vec::new();
    while let Some(row) = rows.next().await? {
        versions.push(row.get::<String>(0)?);
    }
    Ok(versions)
}

/// 1つのマイグレーションの SQL を実行し、適用済みとして記録する
async fn apply(tx: &Transaction, migration: &Migration) -> Result<(), libsql::Error> {
    tx.execute_batch(migration.sql).await?;
    tx.execute(
        "INSERT INTO schema_migrations (version) VALUES (?1)",
        [migration.version],
    )
    .await?;
    Ok(())
}
```

▼ コードを1つずつ分解して解説

解説1: SQL のファイルは、実行ファイルの中に入る

- `MIGRATIONS` の `include_str!("../migrations/0001_users.sql")` は、コンパイルのときに、SQL ファイルの中身を文字列として実行ファイルに埋め込みます。公開した実行ファイルの隣に `migrations/` フォルダが無くても、マイグレーションを流せます（第9巻 UNIT 03 のエラー2）。
- `MIGRATIONS` の並び順が、適用の順番です。ファイル名の番号（0001・0002・0003）は、人が見て分かるように付けているもので、順番を決めているのはこの配列です。

解説2: Db は、Arc で包んだ入口

- `Db` は、libsql の `Database` を `Arc` で包んでいます。`clone()` しても、中の DB は1つのままで、参照の数が増えるだけです（第3巻）。RustNote の `AppState` は、この `Db` を、3つのリポジトリに1つずつ `clone()` して渡しています。
- 接続（`Connection`）は、処理のたびに `connect()` で作ります。先頭のコメントのとおり、ローカルのファイルなら開くだけ、Turso なら HTTP の準備だけなので軽い、という判断です。

解説3: 失敗の理由を2種類に分ける

- `MigrateError` は、「記録簿を準備できなかった」（`Prepare`）と、「この版の適用に失敗した」（`Apply`、失敗した版の名前つき）の2種類です。`#[source]` を付けたので、もとの libsql のエラーも、原因としてつながったまま残ります（第4巻 UNIT 05）。エラー1で、実際の表示を見ます。

例2-2: 同じ DB に、migrate を2回かける

> **▼ このコードがやること（先に日本語で）**：引数で渡した場所に DB ファイルを開き（無ければ作られる）、`db::migrate` を2回続けて呼んで、それぞれが返した「今回適用した版」を表示します。そのあと、記録簿の中身、できた表と索引、`notes` の列の名前を表示します。

```rust
//! RustNote の db::migrate を同じ DB に2回かけて、何が作られ、2回目に何が起きるかを確かめる。
//! DB ファイルの場所を引数で渡す（無ければ作られる）。
//! 実行: cargo run -q --example u02_migrate -- try.db

use libsql::Connection;
use rustnote::db::{self, Db};

/// 1列目の文字列を、行の数だけ集める
async fn strings(conn: &Connection, sql: &str) -> anyhow::Result<Vec<String>> {
    let mut rows = conn.query(sql, ()).await?;
    let mut values = Vec::new();
    while let Some(row) = rows.next().await? {
        values.push(row.get::<String>(0)?);
    }
    Ok(values)
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let path = std::env::args()
        .nth(1)
        .unwrap_or_else(|| "try.db".to_string());
    let db = Db::open_local(&path).await?;

    println!("1回目の migrate: {:?}", db::migrate(&db).await?);
    println!("2回目の migrate: {:?}", db::migrate(&db).await?);

    let conn = db.connect().await?;
    println!(
        "適用済みの版: {:?}",
        strings(
            &conn,
            "SELECT version FROM schema_migrations ORDER BY version"
        )
        .await?
    );
    println!(
        "表: {:?}",
        strings(
            &conn,
            "SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY name"
        )
        .await?
    );
    println!(
        "索引: {:?}",
        strings(
            &conn,
            "SELECT name FROM sqlite_master WHERE type = 'index' AND name LIKE 'idx_%'
```

```
ORDER BY name"
        )
        .await?
    );
    println!(
        "notes の列: {:?}",
        strings(&conn, "SELECT name FROM pragma_table_info('notes')").await?
    );
    Ok(())
}
```

```
cargo run -q --example u02_migrate -- try.db
```

▼ 実行結果

```
1回目の migrate: ["0001_users", "0002_notes", "0003_sessions"]
2回目の migrate: []
適用済みの版: ["0001_users", "0002_notes", "0003_sessions"]
表: ["notes", "schema_migrations", "sessions", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated", "idx_sessions_expires", "idx_sessions_user"]
notes の列: ["id", "user_id", "title", "body", "done", "version", "created_at",
"updated_at"]
```

- 1回目は3つの版がすべて適用され、2回目は `[]`（何も適用しない）でした。RustNote は起動のたびに `migrate` を呼びますが、2回目からは何もしません。何回呼んでも同じ結果になる（**冪等**な）作りです。
- 表には、マイグレーションの4つのほかに `sqlite_sequence` があります。`AUTOINCREMENT` を付けた列の「次に使う番号」を、SQLite が自動で記録する表です。自分で作る必要も、触る必要もありません。

> **用語:** 冪等（べきとう：同じ操作を何回くり返しても、1回したときと同じ結果になること）

もう一歩: 実務に近い形

例2-3: 接続ごとの設定を、libsql の素の接続と比べる

> **▼ このコードがやること（先に日本語で）:** 一時フォルダの DB を `Db::open_local` で開き、`Db::connect` の接続で、3つの設定（`journal_mode`・`foreign_keys`・`busy_timeout`）の値を表示します。同じファイルを、RustNote の `Db` を通さずに、libsql の素の接続でも開いて比べます。最後に、フォルダの中のファイルを表示します。

```rust
//! Db::connect が接続ごとに入れている設定（PRAGMA）を、libsql の素の接続と比べる。
//! 実行: cargo run -q --example u02_pragmas

use libsql::{Builder, Connection, Value};
use rustnote::db::Db;

/// PRAGMA の値を1つ読む（数のときも文字のときもあるので、両方を文字にする）
async fn pragma(conn: &Connection, name: &str) -> anyhow::Result<String> {
    let mut rows = conn.query(&format!("PRAGMA {name}"), ()).await?;
    let row = rows.next().await?.expect("PRAGMA は1行を返すはずです");
    Ok(match row.get_value(0)? {
        Value::Integer(number) => number.to_string(),
        Value::Text(text) => text,
        other => format!("{other:?}"),
    })
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let path = dir.path().join("u02.db");

    // RustNote の Db を通した接続
    let db = Db::open_local(&path).await?;
    let conn = db.connect().await?;
    println!("[Db::connect の接続]");
    for name in ["journal_mode", "foreign_keys", "busy_timeout"] {
        println!("    {name} = {}", pragma(&conn, name).await?);
    }

    // libsql の素の接続（RustNote の設定を通さない）
    let raw = Builder::new_local(&path).build().await?.connect()?;
    println!("[libsql の素の接続]");
    for name in ["journal_mode", "foreign_keys", "busy_timeout"] {
        println!("    {name} = {}", pragma(&raw, name).await?);
    }

    // WAL モードにすると、DB の横に作業用のファイルができる
    let mut files: Vec<String> = std::fs::read_dir(dir.path())?
        .filter_map(|entry| entry.ok())
        .map(|entry| entry.file_name().to_string_lossy().to_string())
        .collect();
    files.sort();
    println!("フォルダの中のファイル: {files:?}");
    Ok(())
}
```

```
cargo run -q --example u02_pragmas
```

▼ 実行結果

```
[Db::connect の接続]
    journal_mode = wal
    foreign_keys = 1
    busy_timeout = 5000
[libsql の素の接続]
    journal_mode = wal
    foreign_keys = 1
    busy_timeout = 0
フォルダの中のファイル: ["u02.db", "u02.db-shm", "u02.db-wal"]
```

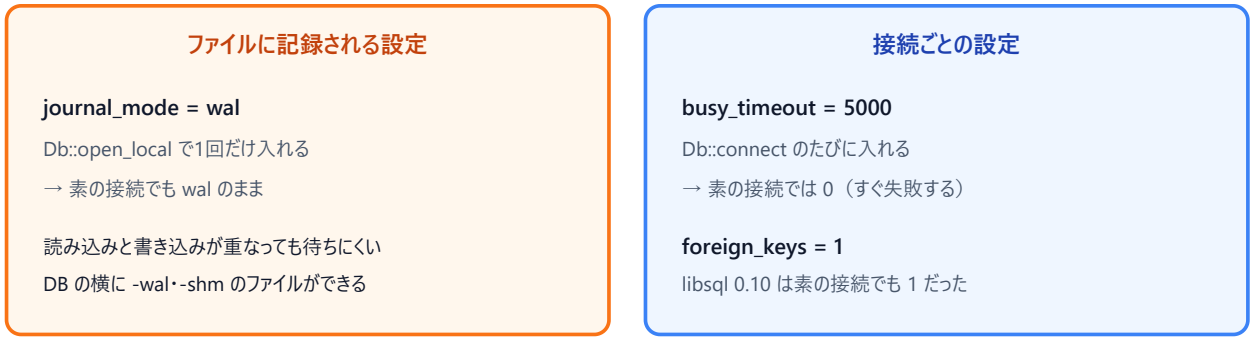


図13-9: DB の設定の2種類。WAL はファイルに記録されるので1回でよく、待ち時間と外部キーは接続ごとに入れる。RustNote は、外部キーも念のため毎回入れている

- **WAL（journal_mode = wal）**：ファイルに記録される設定です。`Db::open_local` で1回入れたので、素の接続でも `wal` のままでした。WAL にすると、DB のファイル（`u02.db`）の横に、`u02.db-wal`（まだ本体に書き込んでいない変更）と `u02.db-shm`（その目次）ができます。RustNote の `.gitignore` が `*.db-wal`・`*.db-shm` を外しているのは、このためです（UNIT 01）。
- **busy_timeout:** 接続ごとの設定です。`Db::connect` の接続は 5000（ミリ秒）、素の接続は 0 でした。0 のままだと、ほかの接続が書き込み中のとき、待たずにすぐ「DB がふさがっている」エラーになります。RustNote は、最大 5 秒は待つようにしています。
- **foreign_keys:** SQLite では、外部キーの確かめは「接続ごとに ON にしないと働かない」決まりです。ところが、この libsql 0.10 では、素の接続でも最初から `1`（ON）でした。RustNote の `Db::connect` は、それでも毎回 `PRAGMA foreign_keys = ON` を入れています。libsql の既定が変わっても外部キーが働くように、そして、コードを読んだ人が「外部キーを守らせている」と分かるように書いてある、と読めます。

よくあるエラーと読み方

この節の実験では、練習用のコピーの `migrations/` のファイルを書き換えます。確かめ終わったら、教材のフォルダから、そのファイルをコピーし直して元に戻してください。

エラー1: マイグレーションが途中で失敗した

`0003_sessions.sql` の最後の行の列の名前を、`expires_at` から `expire_at`（`s` が無い）に打ち間違えて、新しい DB に `migrate` をかけた場合です。

```
CREATE INDEX idx_sessions_expires ON sessions (expire_at);
```

```
cargo run -q --example u02_migrate -- broken.db
```

```
Error: マイグレーション 0003_sessions に失敗しました

Caused by:
    SQLite failure: `no such column: expire_at`
```

- **何が起きたか:** `Error:` の行は、`MigrateError::Apply` の文で、どの版で失敗したか（`0003_sessions`）を教えています。`Caused by:` の下が、もとの libsql のエラーで、「`expire_at` という列がありません」です。
- **何が残ったか:** 失敗したあとの DB の中を、`migrate` をかけずに表示するプログラム（`examples/u02_state.rs`）で調べます。

```rust
//! DB の今の状態（適用済みの版・表・索引）を、migrate をかけずに表示する。
//! 実行: cargo run -q --example u02_state -- try.db

use libsql::Connection;
use rustnote::db::Db;

/// 1列目の文字列を、行の数だけ集める（表が無ければ空にする）
async fn strings(conn: &Connection, sql: &str) -> Vec<String> {
    let Ok(mut rows) = conn.query(sql, ()).await else {
        return Vec::new();
    };
    let mut values = Vec::new();
    while let Ok(Some(row)) = rows.next().await {
        if let Ok(value) = row.get::<String>(0) {
            values.push(value);
        }
    }
    values
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let path = std::env::args()
        .nth(1)
        .unwrap_or_else(|| "try.db".to_string());
    let db = Db::open_local(&path).await?;
    let conn = db.connect().await?;
    println!(
        "適用済みの版: {:?}",
        strings(
            &conn,
            "SELECT version FROM schema_migrations ORDER BY version"
        )
        .await
    );
    println!(
        "表: {:?}",
        strings(
            &conn,
            "SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY name"
        )
        .await
    );
    println!(
        "索引: {:?}",
        strings(
            &conn,
            "SELECT name FROM sqlite_master WHERE type = 'index' AND name LIKE 'idx_%'
ORDER BY name"
```

```
        )
        .await
    );
    Ok(())
}
```

```
cargo run -q --example u02_state -- broken.db
```

```
適用済みの版: ["0001_users", "0002_notes"]
表: ["notes", "schema_migrations", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated"]
```

- 0001 と 0002 は記録されていますが、0003 は記録されていません。そして、`sessions` の表も、その前の行で作ったはずの索引 `idx_sessions_user` も、**ありません**。0003 の SQL は、`sessions` の表と1つ目の索引を作ったところで失敗しましたが、トランザクションの `rollback` で、作りかけの物が全部取り消されたのです。
- **直し方**: SQL ファイルの打ち間違いを直してから、同じ DB にもう一度 `migrate` をかけます。0001 と 0002 は記録簿にあるので飛ばされ、0003 だけが適用されました。

```
1回目の migrate: ["0003_sessions"]
2回目の migrate: []
適用済みの版: ["0001_users", "0002_notes", "0003_sessions"]
表: ["notes", "schema_migrations", "sessions", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated", "idx_sessions_expires", "idx_sessions_user"]
notes の列: ["id", "user_id", "title", "body", "done", "version", "created_at",
"updated_at"]
```

エラー2（エラーが出ない間違い）: 適用済みの SQL ファイルを書き換えた

ノートに「ピン留め」の列 `pinned` を足したくなって、**適用済み**の `0002_notes.sql` の `CREATE TABLE notes` に、1行を書き足した場合です。

```
    body TEXT NOT NULL DEFAULT '',
    pinned INTEGER NOT NULL DEFAULT 0,
```

例2-2で作った、3つの版が適用済みの `try.db` に、`migrate` をかけます。

```
cargo run -q --example u02_migrate -- try.db
```

```
1回目の migrate: []
2回目の migrate: []
適用済みの版: ["0001_users", "0002_notes", "0003_sessions"]
表: ["notes", "schema_migrations", "sessions", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated", "idx_sessions_expires", "idx_sessions_user"]
notes の列: ["id", "user_id", "title", "body", "done", "version", "created_at",
"updated_at"]
```

- **何が起きたか:** 1回目も2回目も `[]` で、`notes` の列に `pinned` はありません。エラーも出ません。`migrate` は、版の**名前**（`0002_notes`）が記録簿にあるかどうかだけを見るので、中身が変わっても、もう一度は流さないのです。
- **なぜ困るのか:** 新しく作った DB（たとえば、別の人の PC や、テストの DB）では、書き換えた 0002 が流れるので、`pinned` があります。

```
cargo run -q --example u02_migrate -- fresh.db
```

```
1回目の migrate: ["0001_users", "0002_notes", "0003_sessions"]
notes の列: ["id", "user_id", "title", "body", "pinned", "done", "version",
"created_at", "updated_at"]
```

- 同じ RustNote なのに、**古い DB には `pinned` が無く、新しい DB にはある**という食い違いが、黙って生まれました。テストは新しい DB で動くので通り、本番の古い DB で初めて「列がありません」のエラーになります。
- **直し方:** 適用済みのファイルは、書き換えません。`0002_notes.sql` を元に戻し、変更は**新しい版**（`0004_…`）として足します（問1）。

RustNote ではここで使う

起動の入口の `src/main.rs` は、DB を開いて、マイグレーションをかけてから、待ち受けを始めます（抜き出しています）。

```
    let db = Db::open(&config)
        .await
        .with_context(|| format!("DB（{}）を開けませんでした", config.database_url))?;
    let applied = db::migrate(&db)
        .await
        .context("マイグレーションに失敗しました")?;
    for version in &applied {
        eprintln!("マイグレーションを適用しました: {version}");
    }
```

- `migrate` が返した「今回適用した版」を、1つずつ表示しています。第12巻 UNIT 01 で、新しい DB で起動したときに `マイグレーションを適用しました: 0001_users` などが3行出たのは、ここです。2回目からは、何も表示されません。
- マイグレーションに失敗したら、`?` で起動をやめます。表がそろっていない DB のまま、リクエストを受け付けることはありません。

テストの道具 `tests/common/mod.rs` も、同じ関数を使っています（抜き出しています）。

```
/// マイグレーション済みの空の DB を作る
pub async fn test_db() -> TestDb {
    let dir = tempfile::tempdir().expect("一時フォルダを作れません");
    let db = Db::open_local(dir.path().join("test.db"))
        .await
        .expect("DB を開けません");
    db::migrate(&db).await.expect("マイグレーションに失敗");
    TestDb { db, _dir: dir }
}
```

- テストごとに、一時フォルダに新しい DB を作り、本番と**同じ** `migrate` をかけます。テスト用に別の `CREATE TABLE` を書かないので、「テストの表と本番の表が違う」ことが起きません。

> **注意:** `0002_notes.sql` のコメントには「`length()` は文字数を数える。Rust 側の `chars().count()` と同じ」とあります。ふつうの文字ではそのとおりですが、第11巻 UNIT 04 の property test で、**NUL**（番号 0 の文字）だけは違うことが分かりました。SQLite の `length()` は NUL より前の文字しか数えないので、NUL で始まる題は、Rust の `NoteTitle::parse` を通っても、この `CHECK` で断られて 500 になります。教材のコードは、まだ直していません（改善点。直し方は、第11巻 UNIT 04 の差分と、巻末の総合演習で扱います）。

練習問題

問1: エラー2の「ピン留め」の列を、正しいやり方で足してください。`0002_notes.sql` は元に戻したうえで、新しい版 `migrations/0004_note_pinned.sql` を作り、`src/db.rs` の `MIGRATIONS` に足します（練習用のコピーの `src/` を書き換える練習です。確かめたら元に戻します）。3つの版が適用済みの `try.db` に `migrate` をかけたら、何が適用されるでしょうか。

問2: `0002_notes.sql` の索引 `idx_notes_user_updated` は、「自分のノートを新しい順に」を速くするためのものです。RustNote の一覧の SQL（`src/repository/notes.rs`）が、本当にこの索引を使っているかを、`EXPLAIN QUERY PLAN` で確かめてください。持ち主の条件が無い SQL とも比べます。

問3: `.env` の `DATABASE_URL` に、まだ無いフォルダの中のファイル（たとえば `data/rustnote.db`）を書いたら、RustNote はどうなるでしょうか。`Db::open_local` で確かめてください。

解答と解説

問1の解答:

新しいファイル `migrations/0004_note_pinned.sql` です。

```
-- 0004: ノートを一覧の上に固定する印（ピン留め）
-- 0（固定しない）か 1（固定する）。今あるノートは 0 になる。
ALTER TABLE notes ADD COLUMN pinned INTEGER NOT NULL DEFAULT 0 CHECK (pinned IN (0,
1));
```

`src/db.rs` の `MIGRATIONS` の最後に、1つ足します。

```
    Migration {
        version: "0004_note_pinned",
        sql: include_str!("../migrations/0004_note_pinned.sql"),
    },
```

```
cargo run -q --example u02_migrate -- try.db
```

```
1回目の migrate: ["0004_note_pinned"]
2回目の migrate: []
適用済みの版: ["0001_users", "0002_notes", "0003_sessions", "0004_note_pinned"]
表: ["notes", "schema_migrations", "sessions", "sqlite_sequence", "users"]
索引: ["idx_notes_user_updated", "idx_sessions_expires", "idx_sessions_user"]
notes の列: ["id", "user_id", "title", "body", "done", "version", "created_at",
"updated_at", "pinned"]
```

- 1回目の `migrate` で、`0004_note_pinned` だけが適用されました。0001〜0003 は記録簿にあるので飛ばされます。`notes` の列の最後に `pinned` が加わりました。`ALTER TABLE ... ADD COLUMN` は、列を最後に足すからです。
- 今あるノートには、`DEFAULT 0` の値が入ります。`NOT NULL` の列を足すときは、`DEFAULT` が無いと、今ある行に入れる値が無いので失敗します。
- この状態で `cargo test` を実行すると、41 件のテストがすべて通りました。新しい DB では 0001〜0004 がすべて流れ、古い DB では 0004 だけが流れて、どちらも同じ形になります。確かめたら、`src/db.rs` を元に戻し、`0004_note_pinned.sql` を消します。

問2の解答:

`examples/u02_q2.rs` です。

```rust
//! 問2の解答: 一覧の SQL が、索引 idx_notes_user_updated を使っているかを EXPLAIN QUERY PLAN で確かめる。
//! 実行: cargo run -q --example u02_q2

use rustnote::db::{self, Db};

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("u02.db")).await?;
    db::migrate(&db).await?;
    let conn = db.connect().await?;

    // src/repository/notes.rs の一覧の SQL と同じ形（列は短くしている）
    let list_sql = r"SELECT id, title FROM notes
        WHERE user_id = ?1 AND (title LIKE ?2 ESCAPE '\' OR body LIKE ?2 ESCAPE '\')
        ORDER BY updated_at DESC, id DESC
        LIMIT ?3 OFFSET ?4";
    for (label, sql) in [
        ("RustNote の一覧", list_sql.to_string()),
        (
            "持ち主の条件が無い一覧",
            "SELECT id, title FROM notes ORDER BY updated_at DESC, id DESC LIMIT 20".to_string(),
        ),
    ] {
        println!("[{label}]");
        let mut rows = conn.query(&format!("EXPLAIN QUERY PLAN {sql}"), ()).await?;
        while let Some(row) = rows.next().await? {
            println!("    {}", row.get::<String>(3)?);
        }
    }
    Ok(())
}
```

```
cargo run -q --example u02_q2
```

```
[RustNote の一覧]
    SEARCH notes USING INDEX idx_notes_user_updated (user_id=?)
[持ち主の条件が無い一覧]
    SCAN notes
    USE TEMP B-TREE FOR ORDER BY
```

- RustNote の一覧の SQL は `SEARCH notes USING INDEX idx_notes_user_updated (user_id=?)` でした。索引を使って、持ち主（`user_id`）のノートだけを探しています。しかも、並べ替えの手順（`USE TEMP B-TREE FOR ORDER BY`）がありません。索引が、はじめから `updated_at DESC, id DESC` の順に並んでいるので、並べ替えが要らないのです。
- 持ち主の条件が無い SQL は、`SCAN notes`（全部の行を読む）と `USE TEMP B-TREE FOR ORDER BY`（読んだあとで並べ替える）でした。ノートが増えるほど遅くなります。索引の列の順番（`user_id` が先）は、「持ち主で絞ってから、新しい順に並べる」という使い方に合わせて決めてあるのです（第7巻 UNIT 04）。
- `r"..."` は、`\` をそのまま書ける Rust の文字列の書き方です。SQL の `ESCAPE '\'` を、`\\` と書かずに済みます。

問3の解答:

`examples/u02_q3.rs` です。

```
//! 問3の解答: まだ無いフォルダの中の DB ファイルを開こうとすると、どうなるか。
//! 実行: cargo run -q --example u02_q3

use rustnote::db::Db;

#[tokio::main]
async fn main() {
    let dir = tempfile::tempdir().expect("一時フォルダを作れません");
    // ファイルは無くてもよいが、フォルダ（data）は作っていない
    let path = dir.path().join("data").join("rustnote.db");
    match Db::open_local(&path).await {
        Ok(_) => println!("開けました"),
        Err(error) => println!("開けません: {error}"),
    }

    // フォルダを作ってから開くと、ファイルは自動で作られる
    std::fs::create_dir(dir.path().join("data")).expect("フォルダを作れません");
    match Db::open_local(&path).await {
        Ok(_) => println!(
            "フォルダを作ってから: 開けました（ファイルがある: {}）",
            path.exists()
        ),
        Err(error) => println!("フォルダを作ってから: 開けません: {error}"),
    }
}
```

```
cargo run -q --example u02_q3
```

```
開けません: Failed to connect to database: `Unable to open connection to local database
...\rustnote.db: 14`
フォルダを作ってから: 開けました（ファイルがある: true）
```

- フォルダ（`data`）が無いと、`Unable to open connection to local database ...\rustnote.db: 14` で開けませんでした。14 は SQLite のエラーの番号で、「ファイルを開けない」（`SQLITE_CANTOPEN`）という意味です。SQLite は、ファイルは自動で作りますが、**フォルダは作りません**。
- フォルダを作ってから開くと、ファイルが自動で作られました。
- RustNote では、このエラーは `main.rs` の `with_context` で「DB（data/rustnote.db）を開けませんでした」と包まれて表示され、起動が止まります。公開するときの `Dockerfile` が、`mkdir -p /data` でフォルダを先に作っているのは、このためです（UNIT 10（準備中））。

まとめ

- RustNote の表は、`migrations/` の番号順の SQL ファイルが作る。`db::migrate` は、未適用の版だけを古い順に、1つずつトランザクションの中で適用し、SQL の実行と記録簿への書き込みを同じトランザクションに入れる。失敗した版の作りかけは、`rollback` で残らない。
- `Db::open_local` は WAL（ファイルに記録される）を1回入れ、`Db::connect` は外部キーと待ち時間（接続ごと）を毎回入れる。libsql 0.10 は素の接続でも外部キーが ON だったが、RustNote は明示している。
- 適用済みの SQL ファイルは書き換えない。古い DB には反映されず、新しい DB とだけ形が変わってしまう。変更は、新しい版として足す。

この巻のまとめ

この版に収録した UNIT 01〜02 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: 版を固定する設定ファイル

- ☐ `Cargo.toml`・`Cargo.lock`・`rust-toolchain.toml`・`rustfmt.toml`・`.gitignore`・`.env.example` の6つについて、読む道具と、決めていることを言える
- ☐ `"=1.53.3"`（その版だけ）と `"1.53.3"`（1.53.3 以上 2.0.0 未満）の違いを、`cargo update -p tokio --dry-run` の結果で説明できる
- ☐ `rust-version`（最低の版）と `rust-toolchain.toml`（このフォルダで使う版）の違いと、`overridden by` の意味を説明できる
- ☐ `--locked` が `Cargo.lock` の書き換えを止めることと、`Dockerfile` と `ci.yml` が `--locked` を付けている理由を説明できる
- ☐ features は依存のつながり全体で足し算されることを、`cargo tree -f "{p} [{f}]"` の出力で確かめられる。自分が使う機能は、自分の `Cargo.toml` に書く

- ☐ `cargo tree -i クレート名` で、あるクレートを誰が使っているかをたどれる（`ring` ← rustls ← hyper-rustls ← libsql）
- ☐ 版が古い（`requires rustc 1.98`）・機能の名前の打ち間違い・`--locked` の食い違い・checksum の食い違い・`#[tokio::main]` の E0433 を読み、直すファイルを決められる
- ☐ `git check-ignore -v` で、ファイルが `.gitignore` のどの決まりで外されるかを確かめられる

UNIT 02: 表を作る SQL と DB の入口

- ☐ `migrations/` の3つの SQL ファイルが作る表・索引・制約と、記録簿の `schema_migrations` を言える
- ☐ `db::migrate` が、未適用の版だけを古い順に、1つずつトランザクションの中で適用し、SQL の実行と記録を同じトランザクションに入れる理由を説明できる
- ☐ `migrate` を2回かけても2回目は何もしない（冪等である）ことを確かめられる
- ☐ WAL（ファイルに記録される）と、`busy_timeout`・`foreign_keys`（接続ごと）の違いを、実際の値で説明できる
- ☐ マイグレーションが途中で失敗したとき、作りかけの表と索引が `rollback` で残らないことと、直したあとに失敗した版だけが流れることを確かめられる
- ☐ 適用済みの SQL ファイルを書き換えてはいけない理由を、古い DB と新しい DB の食い違いで説明し、変更を新しい版として足せる
- ☐ `EXPLAIN QUERY PLAN` で、一覧の SQL が索引を使っているかを確かめられる

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03（設定と起動の順番）・UNIT 04（ドメインの型とエラーの分類）・UNIT 05（リポジトリ）・UNIT 06（ログイン・お知らせ・共有の状態）・UNIT 07（画面の枠・部品・登録とログインの画面）・UNIT 08（ノートの CRUD）・UNIT 09（見た目）・UNIT 10（テスト・Docker・Fly.io・CI）と、巻末の総合演習・解答・全巻の用語集・索引は準備中で、次の更新から順に追加します。それまでの間、RustNote のファイルは、教材のフォルダの `Rust/rustnote/` で直接読めます。どのファイルも、先頭のコメントに役割が書いてあります。

用語ミニ辞典

この巻（UNIT 01〜02）に出てきた用語です。

UNIT 01 の用語

用語	意味
版の固定（pin）	使う道具や部品の版を、1つに決めて書いておくこと
依存クレート	自分のプログラムが使う、ほかの人が作ったクレート。そのクレートが使うクレートも、間接的な依存になる
`Cargo.toml`	パッケージの名前と、使うクレートと版を、人が書くファイル
`Cargo.lock`	cargo が選んだ全クレートの版と checksum を記録するファイル。手で書き換えない
checksum	クレートの中身から計算した指紋（SHA-256）。取ってきた中身が記録と同じかを確かめる
`--locked`	`Cargo.lock` の書き換えが必要なら、ビルドせずにエラーで止める指定
`rust-version`	そのパッケージに必要な、Rust の最低の版
`rust-toolchain.toml`	そのフォルダで使う Rust の版を決めるファイル。rustup が読む
features	クレートの機能のスイッチ。依存のつながり全体で足し算される
`[dev-dependencies]`	テスト・examples でだけ使うクレート。本番の実行ファイルには入らない
`cargo tree`	依存のつながりを木の形で表示する。`-i` で逆向きにたどる
`.gitignore`	Git に入れないファイルの名前の決まり
`.env` / `.env.example`	環境変数を書くファイルと、その見本。`.env` は Git に入れない
E0433	名前（モジュールやクレートの中の物）が見つからないエラー
E0752	`main` 関数に `async` を付けたエラー

UNIT 02 の用語

用語	意味
マイグレーション	表の作り（スキーマ）を変える手順を、番号順のファイルに分けて書いたもの
`schema_migrations`	適用済みのマイグレーションの版の名前と時刻を記録する表
トランザクション	いくつかの変更を、全部やるか全部やめるかの1まとまりにする仕組み。`commit` で確定、`rollback` で取り消す
冪等	同じ操作を何回くり返しても、1回したときと同じ結果になること
`include_str!`	ファイルの中身を、コンパイルのときに文字列として実行ファイルに埋め込むマクロ
WAL（`journal_mode = wal`）	変更をまず別のファイル（`-wal`）に書く方式。読み込みと書き込みが重なっても待ちにくい。ファイルに記録される
`busy_timeout`	ほかの接続が書き込み中のとき、何ミリ秒まで待つか。接続ごとの設定
`PRAGMA foreign_keys`	外部キーの確かめを ON にする SQLite の設定。SQLite では接続ごとに入れる
`sqlite_sequence`	`AUTOINCREMENT` の次の番号を、SQLite が自動で記録する表
`EXPLAIN QUERY PLAN`	SQL を、DB がどう実行するつもりか（索引を使うか、全部読むか）を表示する
`SQLITE_CANTOPEN`（14）	DB のファイルを開けなかったときのエラーの番号。フォルダが無いときなど